

Instal·lació - Configuració
Docker - Kubernetes



Índex

Introducció	2
Estructura de contenidors (Docker)	3
Estructura de xarxa	4
- Taula de Direccionament	4
- Diagrama Xarxa entorn VirtualBox	5
Preparació Entorn Virtual	6
Configuració Inicial Node Worker	6
Configurar Xarxa	6
Actualitzar Ubuntu	7
Instalar Docker	7
Fase 1: docker-compose	9
Estructura de carpetes	9
Arxiu docker-compose	10
- Serveis Principals	10
- Serveis Secundaris	12
- Configuració restant	14
Altres Arxius	14
Diagrames fluxos	15
- Flux 1: Autenticació d'usuaris i seguretat	15
- Flux 2: Consulta de Productes	15
- Flux 3: Creació comanda	16
- Flux 4: Processament asíncron	16
Desplegament	17
Logs	19
Video Demostratiu Fase 1	21
Fase 2: docker-swarm	22
Configuració de Màquines Worker	22
Unió de Workers dins del Swarm	24
Adaptació docker-stack	25
- Penjar imatges a DockerHub	25
- Fitxer docker-stack	27
Configuració TrueNAS amb NFS	30
Desplegament Swarm	34
Video Demostratiu Fase 2	36
Fase 3: Seguretat a Docker Swarm	37
Anàlisi de vulnerabilitats	37
Migració a Docker Secrets	38
Aïllament Xarxes overlay	41
Gestió TLS Docker Swarm	43
Escaneig d'imatges	44
Fase 4: Kubernetes	46
Introducció a Kubernetes	46
Instal·lació k3s	47

Migració d'arxius principals Docker	49
Configuració final Kubernetes	51
- Arxius Service	51
- Arxius configmap	52
- Kubernetes Secrets	53
- Volumes de Kubernetes	54
Problemes	55
- Canvi DNS a Nginx	55
- Canvi Codi PHP al Frontend	55
- Utilització de Port Forwarding per accedir	56
- Pre-Carregament d'imatges a Kubernetes	57
Desplegament Final / Video Demostratiu	58

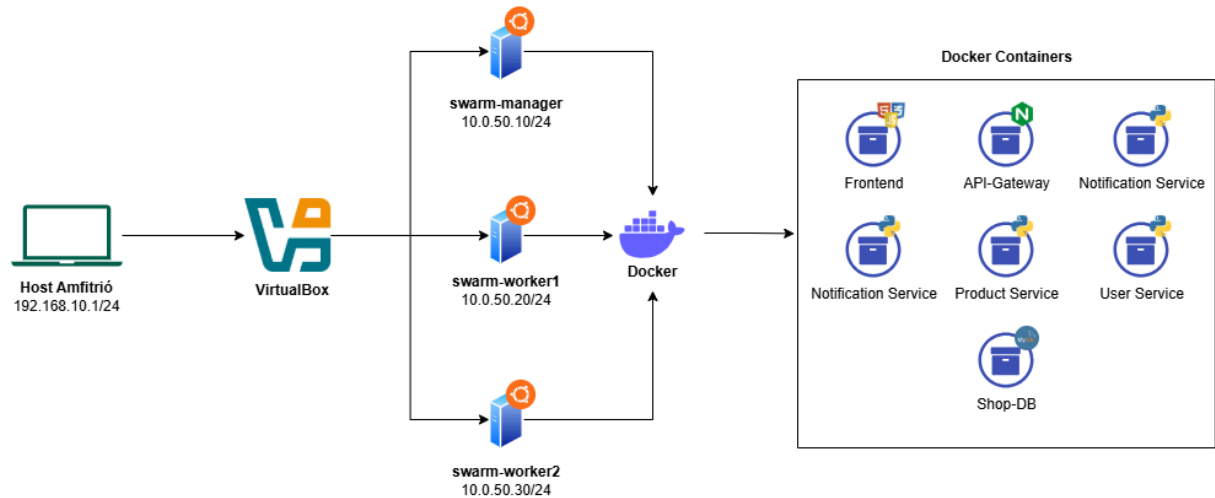
Introducció

Aquest document recull el procés de disseny, desplegament i orquestració d'una plataforma d'e-commerce basada en una arquitectura de microserveis (ShopMicro). L'objectiu principal és abandonar els models de desplegament tradicionals per adoptar solucions escalables i resilients basades en contenidors.

Al llarg de les següents fases, es documenta l'evolució de la infraestructura, des d'un entorn de desenvolupament local inicial fins a una solució orquestrada d'alta disponibilitat:

- **Fase 1: Docker Compose** → Utilitzem un enfocament declaratiu per definir l'estat desitjat de la nostra infraestructura (xarxes, volums i serveis) en un únic fitxer YAML, facilitant la reproduïbilitat enfront del desplegament imperatiu tradicional.
- **Fase 2: Docker Swarm** → Escalem la solució cap a un clúster d'Alta Disponibilitat, demostrant la tolerància a fallades i la distribució de càrrega entre diferents nodes.
- **Fase 3: Seguretat (DevSecOps)** → Implementem polítiques de seguretat, incloent l'ús de Docker Secrets per al xifratge de credencials i l'aïllament de xarxes overlay.
- **Fase 4: Kubernetes** → Finalment, migrem l'orquestració a Kubernetes (k3s) per assolir un entorn professional, escalable i preparat per a producció.

Estructura de contenidors (Docker)



Aquesta és la **estructura que volem seguir de cara al projecte**, utilitzarem 3 màquines virtuals Ubuntu Server 24 amb Docker instal·lat a cadascuna.

Durant la primera fase només utilitzarem la màquina Manager per definir els contenidors que es poden observar a la dreta. A partir de la fase 2 començarem a utilitzar les 3 màquines alhora per aconseguir desplegar un cluster Swarm amb les mateixes imatges que a la primera fase.

Tecnologies utilitzades als contenidors:

- Mysql (Shop-DB)
- Python (microserveis)
- Apache + PHP (frontend)
- Nginx reverse proxy (api-gateway)

Estructura de xarxa

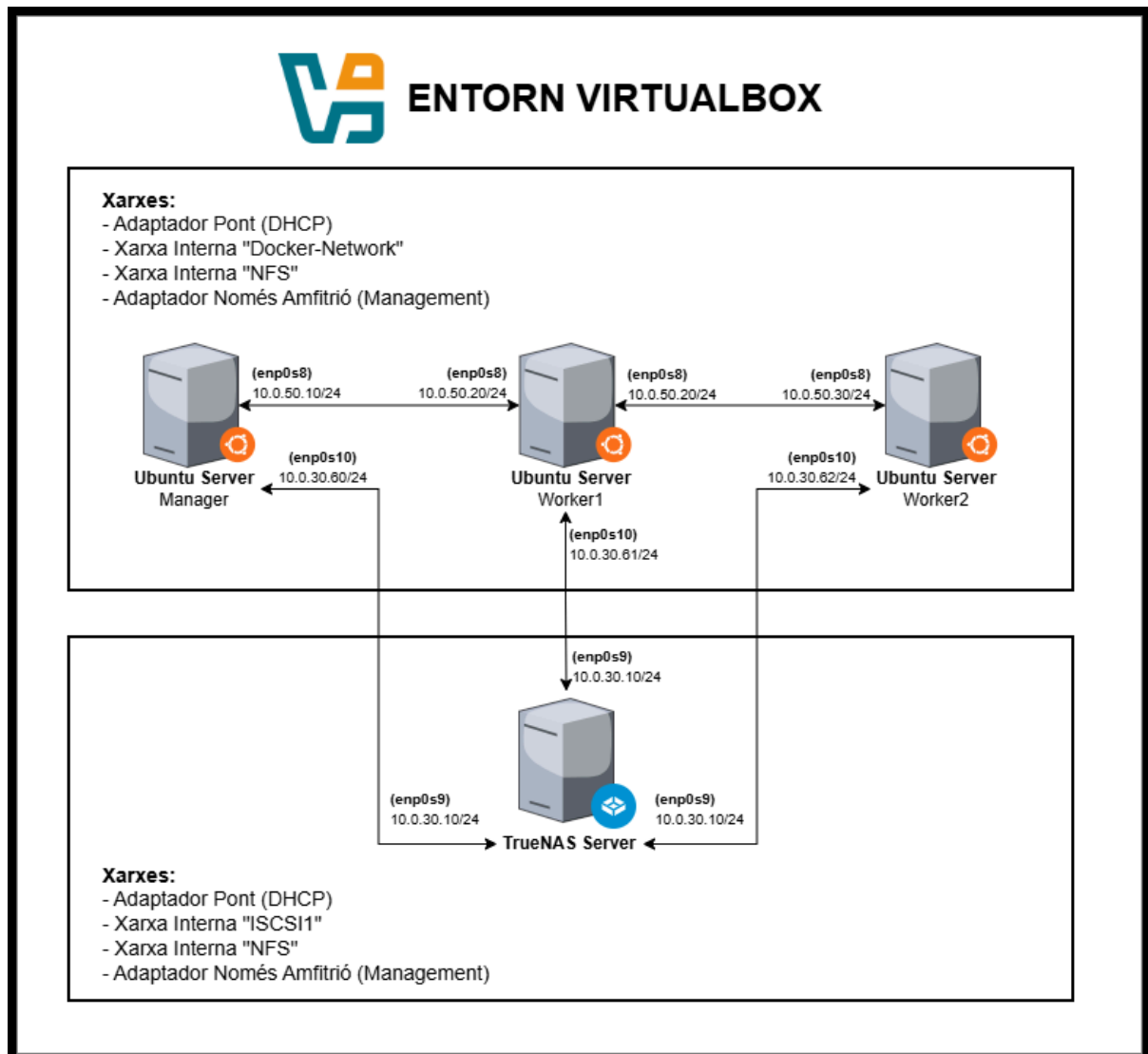
- Taula de Direccionament

En la següent taula es pot comprovar la IP que té cada targeta de xarxa de qualsevol màquina virtual que utilitzem durant aquesta part del projecte.

Servidor	NIC	Xarxa	IP
manager	Adaptador 1 (enp0s3)	DHCP (Internet)	10.0.2.15/24
manager	Adaptador 2 (enp0s8)	Xarxa Interna (docker-network)	10.0.50.10/24
manager	Adaptador 3 (enp0s9)	Xarxa Només Amfitrió (VB)	192.168.10.100/24
manager	Adaptador 4 (enp0s10)	Xarxa Interna (NFS)	10.0.30.60/24
worker1	Adaptador 1 (enp0s3)	DHCP (Internet)	10.0.2.16/24
worker1	Adaptador 2 (enp0s8)	Xarxa Interna (docker-network)	10.0.50.20/24
worker1	Adaptador 3 (enp0s9)	Xarxa Només Amfitrió (VB)	192.168.10.101/24
worker1	Adaptador 4 (enp0s10)	Xarxa Interna (NFS)	10.0.30.61/24
worker2	Adaptador 1 (enp0s3)	DHCP (Internet)	10.0.2.17/24
worker2	Adaptador 2 (enp0s8)	Xarxa Interna (docker-network)	10.0.50.30/24
worker2	Adaptador 3 (enp0s9)	Xarxa Només Amfitrió (VB)	192.168.10.102/24
worker2	Adaptador 4 (enp0s10)	Xarxa Interna (NFS)	10.0.30.62/24
TrueNAS	Adaptador 1 (nic0)	DHCP (Internet)	10.0.2.18/24
TrueNAS	Adaptador 2 (vmbr0)	Xarxa Només Amfitrió (VB)	192.168.10.20/24
TrueNAS	Adaptador 3 (enp0s9)	Xarxa Interna (NFS)	10.0.30.10/24
TrueNAS	Adaptador 4 (enp0s10)	Xarxa Interna (iSCSI)	10.0.40.20/24

- Diagrama Xarxa entorn VirtualBox

En el següent diagrama es pot veure com estan les màquines virtuals connectades entre si dins del nostre entorn de VirtualBox.



En el diagrama es mostren només les xarxes internes per connectar-se entre elles, totes les màquines disposen d'un adaptador NAT (DHCP) per tindre internet i un adaptador només amfitrió per simular una xarxa de "Management" (iDRAC) com als servidors reals.

A més a més, hem decidit implementar també el TrueNAS del projecte de virtualització, amb la finalitat de poder guardar la informació dels volums de Docker allà dins.

Preparació Entorn Virtual

Per realitzar aquesta fase, cal crear les màquines virtuals on s'executarà Docker i Kubernetes per gestionar tots els serveis necessaris per la tenda online.

Hem considerat crear 3 màquines virtuals Ubuntu Desktop 24, una d'elles actuarà com a manager del cluster i les altres actuaran com worker.

Per aquesta primera fase, només utilitzarem la màquina Manager, en la resta de fases començarem a utilitzar les 3 màquines alhora.

Configuració Inicial Node Worker

Configurar Xarxa

```
llucsanchez@swarm-manager: ~
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
# This file is generated from information provided by the datasource.  Changes
# to it will not persist across an instance reboot.  To disable cloud-init's
# network configuration capabilities, write a file
# /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg with the following:
# network: {config: disabled}
network:
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      addresses:
        - 10.0.50.10/24
    enp0s9:
      dhcp4: false
      addresses:
        - 192.168.10.100/24
  version: 2
```

Primer de tot, cal accedir a l'arxiu netplan (ja que ens trobem en un Ubuntu) de la nostra màquina manager i fer un canvi de IP a les interfícies xarxa-interna i adaptador només amfitrió.

```
llucsanchez@swarm-manager: ~
llucsanchez@swarm-manager:~$ sudo nano /etc/netplan/50-cloud-init.yaml
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:d8:ef:8b brd ff:ff:ff:ff:ff:ff
   inet 10.93.254.19/16 metric 100 brd 10.93.255.255 scope global dynamic enp0s3
       valid_lft 86309sec preferred_lft 86309sec
   inet6 fe80::a00:27ff:fed8:ef8b/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:79:63:17 brd ff:ff:ff:ff:ff:ff
   inet 10.0.50.10/24 brd 10.0.50.255 scope global enp0s8
       valid_lft forever preferred_lft forever
   inet6 fe80::a00:27ff:fe79:6317/64 scope link
       valid_lft forever preferred_lft forever
4: enp0s9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:62:ed:88 brd ff:ff:ff:ff:ff:ff
   inet 192.168.10.100/24 brd 192.168.10.255 scope global enp0s9
       valid_lft forever preferred_lft forever
   inet6 fe80::a00:27ff:fe62:ed88/64 scope link
       valid_lft forever preferred_lft forever
llucsanchez@swarm-manager:~$
```

Si tot ha anat correctament, amb la comanda "ip a" ens mostren les IP assignades correctament. **Això s'ha de fer a totes les màquines que utilitzem durant aquesta fase.**

Actualitzar Ubuntu

 llucsanchez@swarm-manager: ~

```
llucsanchez@swarm-manager:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
llucsanchez@swarm-manager:~$
```

També cal actualitzar el sistema operatiu amb l'última versió dels paquets que utilitzarem en aquesta fase, com es veu a la imatge, es troba tot actualitzat.

Instalar Docker

Ara, instal·larem Docker segons la [documentació oficial](#), a les imatges de la part inferior es pot comprovar que instal·lem un certificat des del servidor de docker per afegir un repositori personalitzat (download.docker.com) on agafem els paquets necessaris per tota la instal·lació.

 llucsanchez@swarm-manager: ~

```
llucsanchez@swarm-manager:~$ sudo apt install ca-certificates curl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.8).
curl set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
llucsanchez@swarm-manager:~$ sudo install -m 0755 -d /etc/apt/keyrings
llucsanchez@swarm-manager:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
llucsanchez@swarm-manager:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
llucsanchez@swarm-manager:~$
llucsanchez@swarm-manager:~$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME}-${VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
llucsanchez@swarm-manager:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://es.archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Get:5 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [46.7 kB]
Hit:6 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 95.2 kB in 1s (108 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
llucsanchez@swarm-manager:~$
```

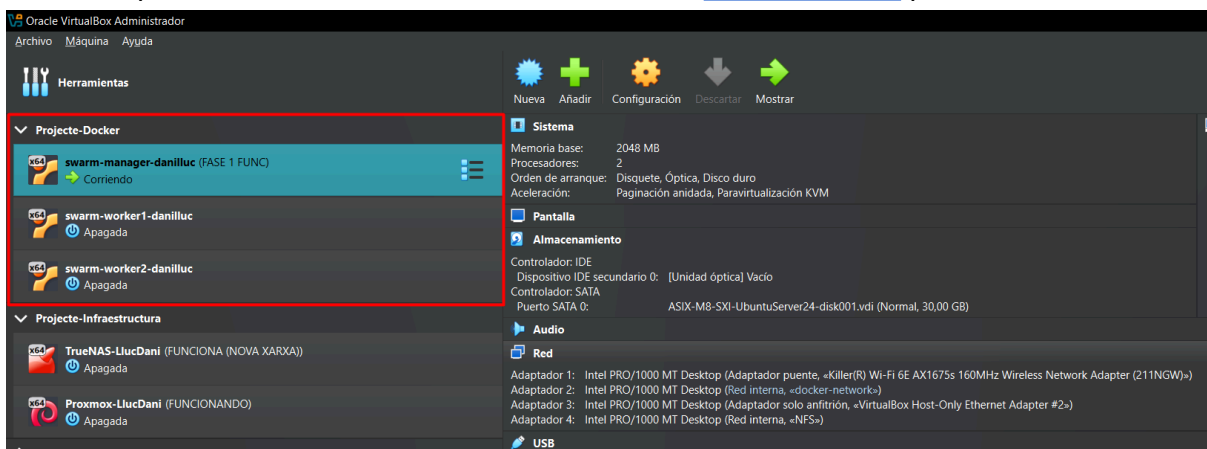
 llucsanchez@swarm-manager: ~

```
llucsanchez@swarm-manager:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker-ce is already the newest version (5:29.3.0-1~ubuntu.24.04~noble).
docker-ce-cli is already the newest version (5:29.3.0-1~ubuntu.24.04~noble).
containerd.io is already the newest version (2.2.2-1~ubuntu.24.04~noble).
docker-buildx-plugin is already the newest version (0.31.1-1~ubuntu.24.04~noble).
docker-compose-plugin is already the newest version (5.1.0-1~ubuntu.24.04~noble).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
llucsanchez@swarm-manager:~$
```

Per últim, afegim a l'usuari que utilitzarem durant tot aquest procés al grup “docker” per poder utilitzar les comandes sense fer “sudo” tota l'estona.

```
llucsanchez@swarm-manager: ~
llucsanchez@swarm-manager:~$ sudo usermod -aG docker llucsanchez
llucsanchez@swarm-manager:~$ cat /etc/group | grep llucsanchez
sudo:x:27:alumne,llucsanchez
llucsanchez:x:1001:
docker:x:988:llucsanchez
llucsanchez@swarm-manager:~$
```

Per les altres 2 màquines workers, cal clonar la màquina que acabem de preparar i fer un canvi d'IP per una diferent en cada NIC, utilitzarem la [taula de xarxes](#) per definir les IP.



Fase 1: docker-compose

Estructura de carpetes

Un cop ja tenim el Docker preparat, cal crear la estructura de carpetes necessària per complir amb els requeriments del projecte i dividir cada servei amb un contenidor.

Hem escollit crear la carpeta arrel “shopmicro” dins de /opt, dins d'aquesta carpeta crearem una per cada contenidor que utilitzem, amb això aconseguim segmentar correctament els arxius de cada contenidor i millorem considerablement la organització del projecte.

 llucsanchez@swarm-manager: /opt/shopmicro

```
llucsanchez@swarm-manager:/opt$ sudo mkdir shopmicro
llucsanchez@swarm-manager:/opt$ sudo chown -R llucsanchez:llucsanchez shopmicro
llucsanchez@swarm-manager:/opt$ cd shopmicro
llucsanchez@swarm-manager:/opt/shopmicro$ mkdir api-gateway db-files frontend notification-service order-service product-service user-service
llucsanchez@swarm-manager:/opt/shopmicro$ ls -l
total 28
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 17 15:58 api-gateway
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 17 15:58 db-files
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 17 15:58 frontend
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 17 15:58 notification-service
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 17 15:58 order-service
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 17 15:58 product-service
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 17 15:58 user-service
llucsanchez@swarm-manager:/opt/shopmicro$
```

 llucsanchez@swarm-manager: /opt

```
llucsanchez@swarm-manager:/opt/shopmicro$ cd ..
llucsanchez@swarm-manager:/opt$ tree shopmicro
shopmicro
├── api-gateway
├── db-files
├── frontend
├── notification-service
├── order-service
├── product-service
└── user-service

8 directories, 0 files
llucsanchez@swarm-manager:/opt$
```

Amb aquesta estructura ja podem començar a treballar en el projecte, si és necessari, en un futur podem crear més carpetes segons les necessitats.

Arxiu docker-compose

Ara, ja ho tenim tot preparat per poder arrencar amb la primera part d'aquesta fase, l'arxiu **docker-compose**. En aquest cal definir la configuració de cada contenidor (microservei).

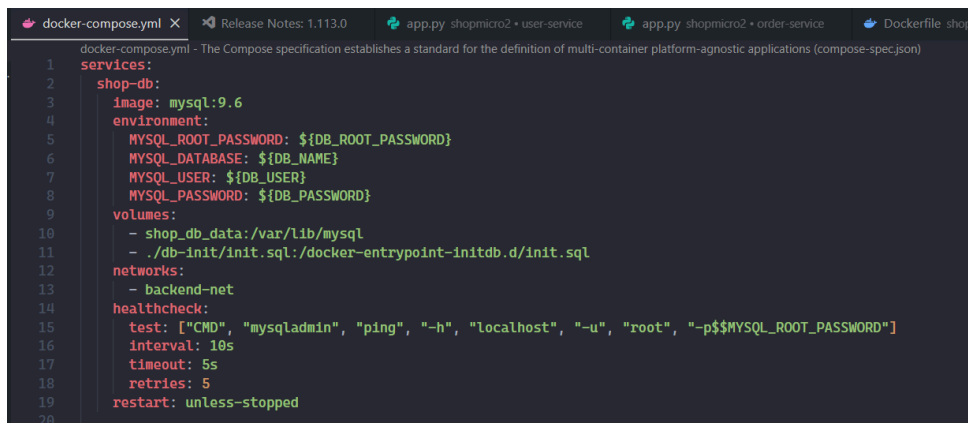
*** Per no tindre que posar moltes captures de l'arxiu en aquest document, adjuntem un enllaç a GitHub on es pot visualitzar tot l'arxiu.*

[Enllaç](#) 

Aquest arxiu l'he dividit en 3 parts: serveis principals, serveis secundaris (*endpoints*) i configuració restant (*volums i xarxes*).

- Serveis Principals

En la part de serveis principals, podem trobar els contenidors de **MySQL**, **Redis** i **RabbitMQ**. Considero que aquest son els serveis fundamentals i més difícils de configurar en aquesta fase, gràcies a aquests contenidors, tots els altres poden funcionar sense cap problema.



```
1 services:
2   shop-db:
3     image: mysql:9.6
4     environment:
5       MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
6       MYSQL_DATABASE: ${DB_NAME}
7       MYSQL_USER: ${DB_USER}
8       MYSQL_PASSWORD: ${DB_PASSWORD}
9     volumes:
10      - shop_db_data:/var/lib/mysql
11      - ./db-init/init.sql:/docker-entrypoint-initdb.d/init.sql
12     networks:
13      - backend-net
14     healthcheck:
15       test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD}"]
16       interval: 10s
17       timeout: 5s
18       retries: 5
19       restart: unless-stopped
20
```

Exemple amb MySQL

Com es veu a la captura del codi, la configuració d'aquests contenidors és senzilla, primer definim la **imatge oficial de MySQL** (mysql:9.6), aquesta és l'última versió que he pogut descarregar en la web oficial de [Docker Hub](#).

Tot seguit, definim les **variables d'entorn** necessàries per el correcte funcionament de la imatge, les variables les guardem en un arxiu a part (.env) que, per motius de seguretat, no hem posat a GitHub. En aquest arxiu es troben els usuaris/credencials de cada servei.

A continuació, definim els **volums** que utilitzarem en aquest contenidor. He creat un per guardar tota la informació relacionada amb la BBDD i un d'altre per poder utilitzar l'arxiu d'inici per definir tota la estructura de la BBDD (init.sql). Aquest també el podem consultar al [GitHub](#).

Just a sota, definim la **xarxa** que volem utilitzar a l'apartat "networks", de moment he introduït "backend-net", l'estructura de xarxes utilitzada està explicada a la segona part (*serveis secundaris*).

Per finalitzar definim un **healthcheck** per comprovar que la BBDD està en funcionament un cop arranquem el contenidor. Cada servei disposa de la seva pròpia forma de fer el healthcheck, en el cas de MySQL es fa amb la comanda `"mysqladmin ping -h localhost -u root -p {contrasenya}"`

Si el contenidor no aconsegueix arrencar, ho tornarà a intentar gràcies al paràmetre `"restart: unless-stopped"`.

Aquesta estructura l'he replicat exactament igual amb els serveis de Redis i RabbitMQ, així complim amb els requeriments d'aquesta primera fase.

- Serveis Secundaris

En aquesta part podem trobar els altres serveis dedicats a la part de frontend, també molt importants per el correcte funcionament de la web:

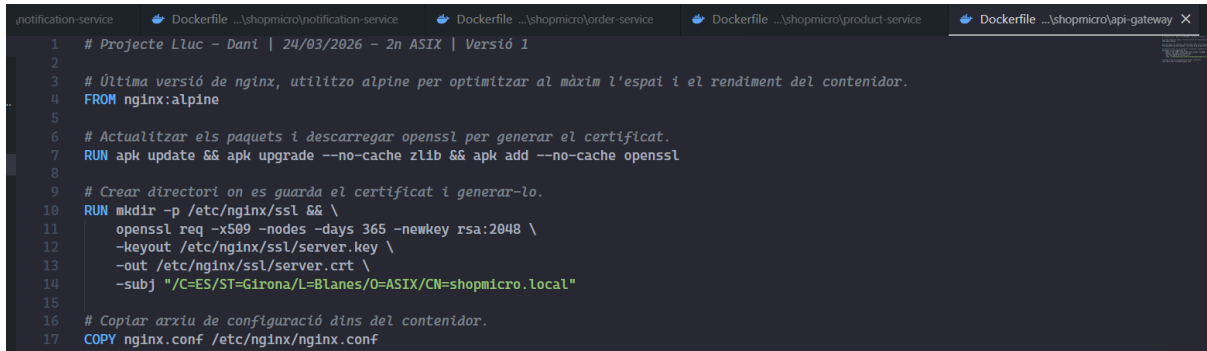
- **frontend:** Servidor web (Apache + PHP) que lliura la interfície gràfica d'usuari (UI) i gestiona les sessions d'autenticació de forma segura.
- **api-gateway:** Servidor Nginx actuant com a Reverse Proxy i SSL Termination. És l'únic punt d'entrada de la infraestructura, xifrant les comunicacions en HTTPS i redirigint el trànsit cap als microserveis interns segons la ruta sol·licitada.
- **product-service:** Microservei desenvolupat en Python (Flask) que gestiona el catàleg de la botiga. Implementa una lògica de memòria cau (Caching) connectant-se a Redis per retornar les dades en mil·lisegons, i a MySQL com a magatzem persistent.
- **order-service:** Microservei transaccional en Python. S'encarrega d'enregistrar les comandes a MySQL, restar l'estoc, invalidar la memòria cau de Redis i publicar esdeveniments asíncrons a RabbitMQ.
- **user-service:** Microservei d'autenticació en Python encarregat de verificar i gestionar les credencials dels usuaris. Realitza la generació i validació de Hashes criptogràfics per evitar l'ús de contrasenyes en text pla.
- **notification-service:** Microservei tipus Worker (segon pla). A diferència dels altres, no rep peticions web, sinó que es dedica exclusivament a escoltar la cua de RabbitMQ per processar els missatges de forma desatesa.

```
api-gateway:  
  build: ./api-gateway  
  ports:  
    - "80:80"  
    - "443:443"  
  networks:  
    - frontend-net  
  volumes:  
    - ./logs/nginx:/var/log/nginx  
  depends_on:  
    - frontend  
    - product-service  
    - order-service  
    - user-service  
  restart: unless-stopped
```

Exemple amb el contenidor "api-gateway"

La definició d'aquests contenidors és molt similar a la que hem fet als serveis principals.

Primer de tot, podem observar que en tots els contenidors hem utilitzat un arxiu **Dockerfile** propi creat per nosaltres per tal de construir les imatges personalitzades, en aquests podem trobar un patró molt similar entre ells:



```
1 # Projecte Lluc - Dani | 24/03/2026 - 2n ASIX | Versió 1
2
3 # Última versió de nginx, utilitzo alpine per optimitzar al màxim l'espai i el rendiment del contenidor.
4 FROM nginx:alpine
5
6 # Actualitzar els paquets i descarregar openssl per generar el certificat.
7 RUN apk update && apk upgrade --no-cache zlib && apk add --no-cache openssl
8
9 # Crear directori on es guarda el certificat i generar-lo.
10 RUN mkdir -p /etc/nginx/ssl && \
11     openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
12     -keyout /etc/nginx/ssl/server.key \
13     -out /etc/nginx/ssl/server.crt \
14     -subj "/C=ES/ST=Girona/L=Blanes/O=ASIX/CN=shopmicro.local"
15
16 # Copiar arxiu de configuració dins del contenidor.
17 COPY nginx.conf /etc/nginx/nginx.conf
```

En totes hem definit l'**imatge** de cada servei que volem utilitzar, en el cas de Nginx hem decidit implementar la versió "alpine", aquesta redueix considerablement l'espai i és més eficient.

Tot seguit indiquem l'**acció** que volem realitzar al arrencar el contenidor, amb Nginx el que fem és crear un certificat SSL en el moment pel tràfic HTTPS.

Per finalitzar, **copiem** l'arxiu de configuració necessari (si cal) per cada microservei, tots els arxius que hem comentat els podem veure més amb detall en el nostre [GitHub](#), a més a més, dins de cada arxiu hem posat comentaris per explicar el que fa cada part.

El motiu principal pel qual hem decidit **dividir-ho tot en Dockerfiles** és per millorar la gestió i configuració de cada microservei, això ens ajuda a desglossar millor el projecte i, si en algun cas falla alguna cosa, poder trobar la configuració al moment i solucionar-ho.

Un cop ja tenim la imatge que volem utilitzar al contenidor, indicarem els **ports** necessaris per el funcionament del servei (*això no ho definim a tots els contenidors*), en el cas de Nginx utilitzem el 80 i 443 per capturar el tràfic WEB.

En aquest cas, la **xarxa** que hem definit és frontend-net. En el nostre arxiu docker-compose hem definit 2 xarxes en total: **frontend i backend**. L'objectiu és segmentar correctament cada contenidor per millorar la seguretat dels serveis, més endavant implementarem una 3a xarxa només per la API (swarm).

A la part de **volums** ho hem definit únicament per guardar els Logs en una carpeta específica per si ho volem monitorar en un futur.

Per acabar, utilitzem el paràmetre "**depends_on**" per dir-li al Docker quins serveis necessitem que s'ajuequin abans d'altres, això va variant entre contenidors depenent de les necessitats de cadascun.

També implementem un "restart" perquè el contenidor es reiniciï constantment en cas d'aturada.

- Configuració restant

En aquesta part, simplement volem recalcar que a la part final del docker-compose hem definit els volums i les xarxes que hem utilitzat en tots els serveis:

```
volumes:
  shop_db_data:
  rabbitmq_data:

networks:
  frontend-net:
    driver: bridge
  backend-net:
    driver: bridge
```

Com es veu a la imatge, hem definit 2 volums persistents de Docker amb la informació de la BBDD i RabbitMQ, en aquests guardem totes les dades que necessitem si o si al reiniciar l'aplicació.

A la part de les xarxes, com hem explicat anteriorment per el moment utilitzarem 2, aquestes en mode bridge.

El **mode Bridge** defineix una xarxa privada interna (*Switch Virtual*) dins de Docker on els contenidors es poden comunicar entre ells. La part més important és que cap contenidor que estigui en aquesta xarxa es pot comunicar amb altres (*com el Backend*).

Altres Arxius

Per a que tots els contenidors funcionin, depenen d'arxius de configuració/execució per poder funcionar de la manera que necessitem i complir amb els requeriments de la fase.

Per exemple:

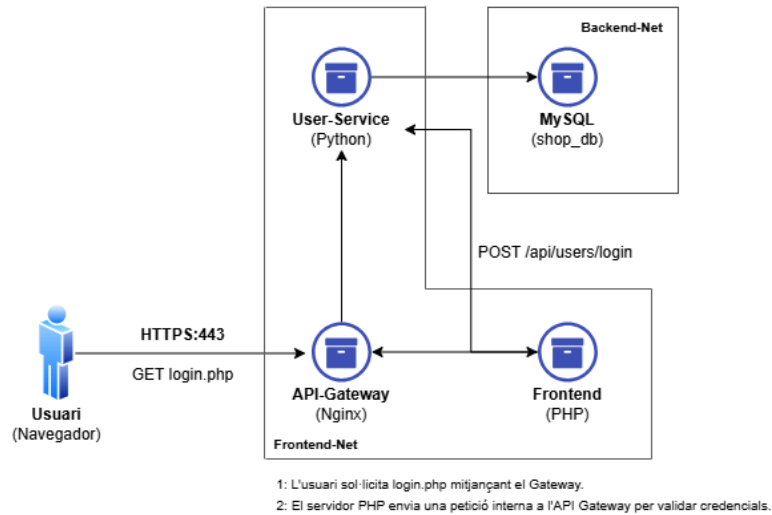
- A la part de "frontend" tenim tots els arxius PHP+CSS per definir la UI (User-Interface) de la web.
- A la part de "api-gateway" tenim l'arxiu de configuració de nginx (*nginx.conf*).
- A la part dels microserveis fets amb Python, tenim l'arxiu .py per poder executar-lo al desplegar la solució i els paquets necessaris per funcionar (*pip install*).

Tots aquests arxius es poden consultar en cada carpeta del contenidor dins del nostre [GitHub](#). 

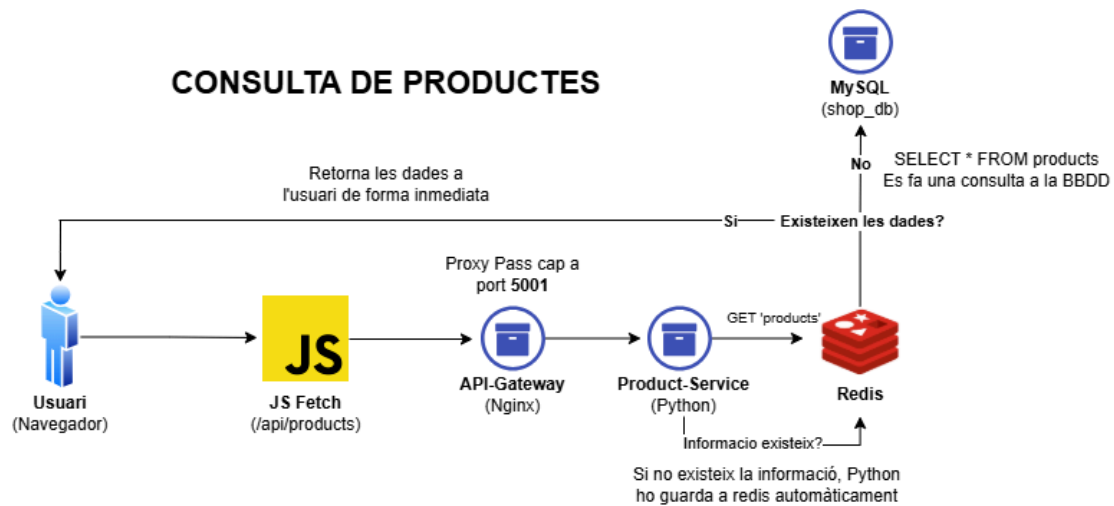
Diagrames fluxos

- Flux 1: Autenticació d'usuaris i seguretat

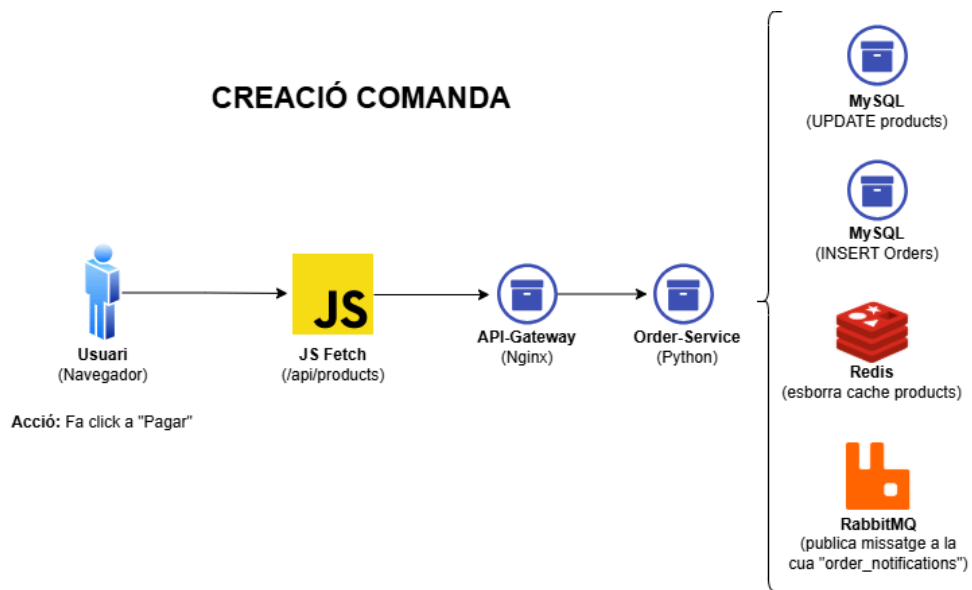
AUTENTICACIÓ I SEGURETAT



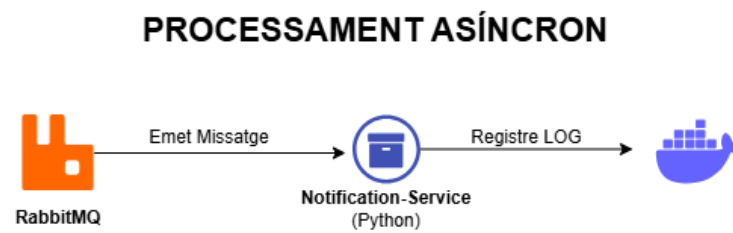
- Flux 2: Consulta de Productes



- Flux 3: Creació comanda



- Flux 4: Processament asíncron



Desplegament

Un cop ja hem tenim la nostra estructura correctament definida, en el node Manager que hem configurat al principi cal desplegar-ho amb docker-compose.

En el nostre cas, obtindrem les dades directament del nostre repositori de GitHub des de la terminal d'Ubuntu Server.

- Comanda "git clone" per obtenir les dades directament del nostre repositori.

```
llucsanchez@swarm-manager: /opt/desplegament/projecte-lluc-dani/docker-kubernetes/shopmicro
llucsanchez@swarm-manager: /opt/desplegament$ git clone https://github.com/DaniRuizP/projecte-lluc-dani
Cloning into 'projecte-lluc-dani'...
remote: Enumerating objects: 97, done.
remote: Counting objects: 100% (97/97), done.
remote: Compressing objects: 100% (67/67), done.
remote: Total 97 (delta 23), reused 81 (delta 13), pack-reused 0 (from 0)
Receiving objects: 100% (97/97), 33.75 MiB | 2.24 MiB/s, done.
Resolving deltas: 100% (23/23), done.
llucsanchez@swarm-manager: /opt/desplegament$ cd projecte-lluc-dani/docker-kubernetes/shopmicro/
llucsanchez@swarm-manager: /opt/desplegament/projecte-lluc-dani/docker-kubernetes/shopmicro$ ls -l
total 32
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Apr  9 17:46 api-gateway
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Apr  9 17:46 db-init
-rw-rw-r-- 1 llucsanchez llucsanchez 3104 Apr  9 17:46 docker-compose.yml
drwxrwxr-x 3 llucsanchez llucsanchez 4096 Apr  9 17:46 frontend
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Apr  9 17:46 notification-service
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Apr  9 17:46 order-service
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Apr  9 17:46 product-service
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Apr  9 17:46 user-service
llucsanchez@swarm-manager: /opt/desplegament/projecte-lluc-dani/docker-kubernetes/shopmicro$
```

- Comanda "docker compose up -d --build" per desplegar tots els contenidors amb les imatges Dockerfile creades per nosaltres.

**** Abans de tot cal copiar l'arxiu .env amb les variables d'entorn corresponents per a que tot funcioni.**

```
llucsanchez@swarm-manager: /opt/desplegament/projecte-lluc-dani/docker-kubernetes/shopmicro
llucsanchez@swarm-manager: /opt/desplegament/projecte-lluc-dani/docker-kubernetes/shopmicro$ cp /opt/shopmicro/.env .
llucsanchez@swarm-manager: /opt/desplegament/projecte-lluc-dani/docker-kubernetes/shopmicro$ docker compose up -d --build
[+] Building 1.3s (10/13)
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 3.41kB                                          0.0s
=> [frontend internal] load build definition from Dockerfile            0.0s
=> => transferring Dockerfile: 190B                                       0.0s
=> [product-service internal] load build definition from Dockerfile     0.1s
=> => transferring Dockerfile: 478B                                       0.0s
=> [api-gateway internal] load build definition from Dockerfile         0.1s
=> => transferring Dockerfile: 758B                                       0.0s
=> [order-service internal] load build definition from Dockerfile       0.1s
=> => transferring Dockerfile: 478B                                       0.0s
=> [user-service internal] load build definition from Dockerfile        0.1s
=> => transferring Dockerfile: 478B                                       0.0s
=> [notification-service internal] load build definition from Dockerfile 0.1s
=> => transferring Dockerfile: 478B                                       0.0s
=> [order-service internal] load metadata for docker.io/library/python:3.14-alpine 0.8s
=> [api-gateway internal] load metadata for docker.io/library/nginx:alpine 0.8s
=> [frontend internal] load metadata for docker.io/library/php:8.2-apache 0.8s
=> [auth library/python] pull token for registry-1.docker.io           0.0s
=> [auth library/nginx] pull token for registry-1.docker.io           0.0s
=> [auth library/php] pull token for registry-1.docker.io              0.0s
```

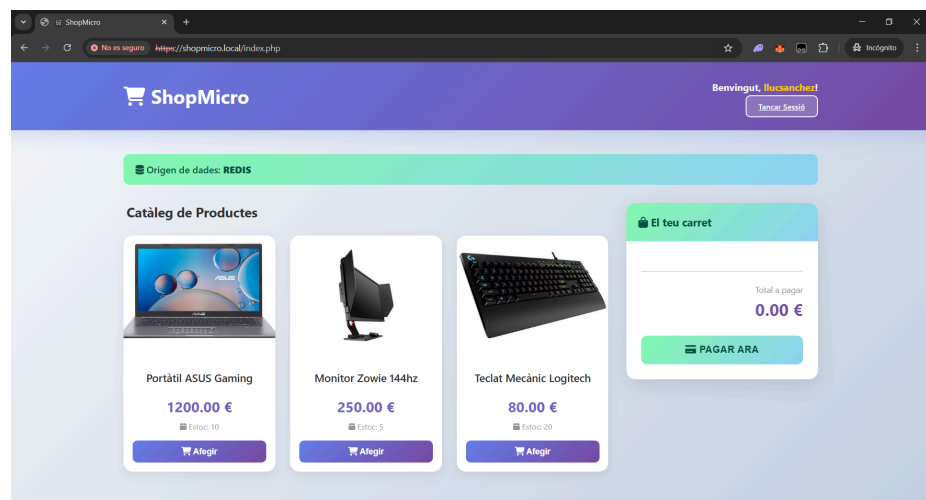
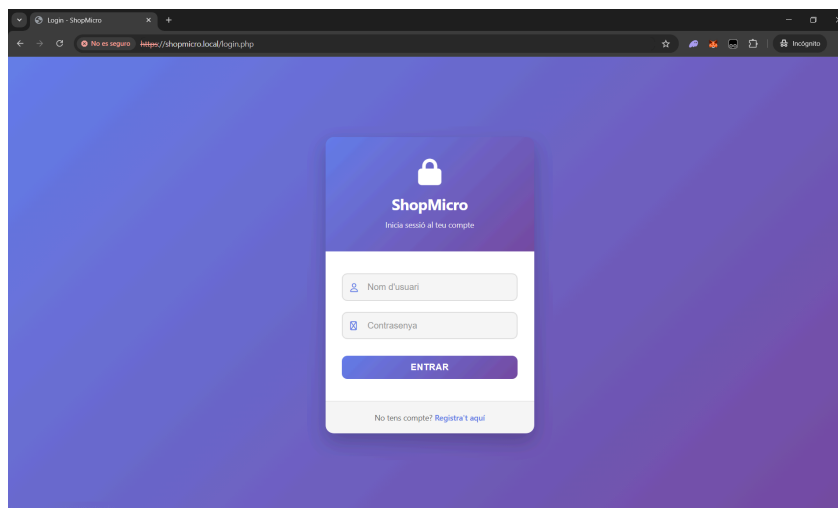
Les comandes dels Dockerfile s'executen correctament:

```
[product-service 4/4] RUN pip install -r requirements.txt              13.7s
=> # Collecting Flask (from -r requirements.txt (line 1))
=> # Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
=> # Collecting redis (from -r requirements.txt (line 2))
=> # Downloading redis-7.4.0-py3-none-any.whl.metadata (12 kB)
=> # Collecting pymysql (from -r requirements.txt (line 3))
=> # Downloading pymysql-1.1.2-py3-none-any.whl.metadata (4.3 kB)
[notification-service 4/4] RUN pip install -r requirements.txt        13.7s
=> # Collecting pika (from -r requirements.txt (line 1))
=> # Downloading pika-1.3.2-py3-none-any.whl.metadata (13 kB)
=> # Downloading pika-1.3.2-py3-none-any.whl (155 kB)
[order-service 4/4] RUN pip install -r requirements.txt               13.7s
=> # Collecting pymysql (from -r requirements.txt (line 2))
=> # Downloading pymysql-1.1.2-py3-none-any.whl.metadata (4.3 kB)
=> # Collecting pika (from -r requirements.txt (line 3))
=> # Downloading pika-1.3.2-py3-none-any.whl.metadata (13 kB)
=> # Collecting redis (from -r requirements.txt (line 4))
=> # Downloading redis-7.4.0-py3-none-any.whl.metadata (12 kB)
[user-service 4/4] RUN pip install -r requirements.txt                13.7s
=> # Collecting Werkzeug (from -r requirements.txt (line 1))
=> # Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
=> # Collecting blinker>=1.9.0 (from Flask->-r requirements.txt (line 1))
=> # Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
=> # Collecting click>=8.1.3 (from Flask->-r requirements.txt (line 1))
=> # Downloading click-8.3.2-py3-none-any.whl.metadata (2.6 kB)
```

Els contenidors s' aixequen tots sense cap problema i funcionen correctament:

```
[*] up 19/19
Image shopmicro-product-service      Built      2.0s
Image shopmicro-user-service         Built      2.0s
Image shopmicro-frontend             Built      2.0s
Image shopmicro-order-service        Built      2.0s
Image shopmicro-api-gateway          Built      2.0s
Image shopmicro-notification-service Built      2.0s
Network shopmicro_frontend-net       Created    0.0s
Network shopmicro_backend-net        Created    0.0s
Volume shopmicro_shop_db_data        Created    0.0s
Volume shopmicro_rabbitmq_data       Created    0.0s
Container shopmicro-message-queue-1   Healthy    11.6s
Container shopmicro-shop-db-1         Healthy    10.6s
Container shopmicro-cache-1           Healthy    11.1s
Container shopmicro-frontend-1        Started    0.6s
Container shopmicro-user-service-1    Started    11.2s
Container shopmicro-order-service-1   Started    11.8s
Container shopmicro-notification-service-1 Started    11.8s
Container shopmicro-product-service-1 Started    11.2s
Container shopmicro-api-gateway-1     Started    12.0s
llucsanchez@swarm-manager:/opt/desplegament/projecte-lluc-dani/docker-kubernetes/shopmicro$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
af755a6dd502   shopmicro-api-gateway              "/docker-entrypoint..." 14 seconds ago Up 2 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
9850eac8d990   shopmicro-product-service          "python -u app.py"        15 seconds ago Up 3 seconds
56581c338e0a   shopmicro-notification-service     "python -u app.py"        15 seconds ago Up 2 seconds
a870cd65633d   shopmicro-order-service            "python -u app.py"        15 seconds ago Up 2 seconds
0d9a88e5fc2c   shopmicro-user-service             "python -u app.py"        15 seconds ago Up 3 seconds
34f5a70480c9   rabbitmq:4.2.5-management-alpine  "docker-entrypoint.s..." 15 seconds ago Up 14 seconds (healthy) 4369/tcp, 5671-5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
23d742c9e57e   redis:8.6.1-alpine                "docker-entrypoint.s..." 15 seconds ago Up 14 seconds (healthy) 6379/tcp
c991fc5a715e   shopmicro-frontend                "docker-php-entrypoi..." 15 seconds ago Up 14 seconds        80/tcp
647eeb9858bf   mysql:9.6                          "docker-entrypoint.s..." 15 seconds ago Up 14 seconds (healthy) 3306/tcp, 33060/tcp
llucsanchez@swarm-manager:/opt/desplegament/projecte-lluc-dani/docker-kubernetes/shopmicro$
```

- Resultat final de la Web en funcionament



Logs

Quan ja ho tenim tot desplegat correctament, podem observar els logs que s'han generat amb la comanda “docker compose logs -f”, això ens serveix per comprovar que no hi ha cap error a qualsevol contenidor:

```
llucsanchez@swarm-manager: /opt/shopmicro
message-queue-1 2026-03-24 14:41:43.403821+00:00 [info] <0.209.0> Successfully set permissions for user 'guest' in virtual host '/' to '.*', '.*', '.*'
message-queue-1 2026-03-24 14:41:43.403787+00:00 [info] <0.209.0> Running boot step rabbit_observer_cli defined by app rabbit
message-queue-1 2026-03-24 14:41:43.403797+00:00 [info] <0.209.0> Running boot step rabbit_core_metrics_gc defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404061+00:00 [info] <0.209.0> Running boot step background_gc defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404120+00:00 [info] <0.209.0> Running boot step rabbit_observer_cli_classic_queues defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404145+00:00 [info] <0.209.0> Running boot step rabbit_observer_cli_quorum_queues defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404159+00:00 [info] <0.209.0> Running boot step routing_ready defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404167+00:00 [info] <0.209.0> Running boot step pre_flight defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404176+00:00 [info] <0.209.0> Running boot step notify_cluster defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404194+00:00 [info] <0.209.0> Running boot step networking defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404208+00:00 [info] <0.209.0> Running boot step rabbit_quorum_queue_periodic_membership_reconciliation defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404307+00:00 [info] <0.209.0> Running boot step definition_import_worker_pool defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404324+00:00 [info] <0.263.0> Starting worker pool 'definition_import_pool' with 2 processes in it
message-queue-1 2026-03-24 14:41:43.404401+00:00 [info] <0.209.0> Running boot step cluster_name defined by app rabbit
message-queue-1 2026-03-24 14:41:43.404440+00:00 [info] <0.209.0> Initialising internal cluster ID to 'rabbitmq-cluster-id-NALmM_f-xcSpXj2f4k72g'
message-queue-1 2026-03-24 14:41:43.409711+00:00 [info] <0.209.0> Running boot step virtual_host_reconciliation defined by app rabbit
message-queue-1 2026-03-24 14:41:43.409901+00:00 [info] <0.209.0> Running boot step direct_client defined by app rabbit
message-queue-1 2026-03-24 14:41:43.410021+00:00 [info] <0.209.0> Running boot step logger_exchange defined by app rabbit
message-queue-1 2026-03-24 14:41:43.412493+00:00 [info] <0.209.0> Running boot step cluster_tags defined by app rabbit
message-queue-1 2026-03-24 14:41:43.414696+00:00 [info] <0.209.0> Running boot step rabbit_management_load_definitions defined by app rabbitmq_management
message-queue-1 2026-03-24 14:41:43.414830+00:00 [info] <0.435.0> Resetting node maintenance status
message-queue-1 2026-03-24 14:41:43.529738+00:00 [warning] <0.457.0> Deprecated features: 'management_metrics_collection': Feature 'management_metrics_collection' is deprecated.
message-queue-1 2026-03-24 14:41:43.529738+00:00 [warning] <0.457.0> By default, this feature can still be used for now.
message-queue-1 2026-03-24 14:41:43.529738+00:00 [warning] <0.457.0> Its use will not be permitted by default in a future minor RabbitMQ version and the feature will be removed fr
on a future major RabbitMQ version; actual versions to be determined.
message-queue-1 2026-03-24 14:41:43.529738+00:00 [warning] <0.457.0> To continue using this feature when it is not permitted by default, set the following parameter in your config
uration:
message-queue-1 2026-03-24 14:41:43.529738+00:00 [warning] <0.457.0> "deprecated_features.permit_management_metrics_collection = true"
message-queue-1 2026-03-24 14:41:43.529738+00:00 [warning] <0.457.0> To test RabbitMQ as if the feature was removed, set this in your configuration:
message-queue-1 2026-03-24 14:41:43.567955+00:00 [warning] <0.457.0> "deprecated_features.permit_management_metrics_collection = false"
message-queue-1 2026-03-24 14:41:43.568156+00:00 [info] <0.493.0> Management plugin: HTTP (non-TLS) listener started on port 15672
message-queue-1 2026-03-24 14:41:43.568156+00:00 [info] <0.521.0> Statistics database started.
message-queue-1 2026-03-24 14:41:43.568216+00:00 [info] <0.520.0> Starting worker pool 'management_worker_pool' with 3 processes in it
message-queue-1 2026-03-24 14:41:43.586255+00:00 [info] <0.532.0> Prometheus metrics: HTTP (non-TLS) listener started on port 15692
message-queue-1 2026-03-24 14:41:43.586354+00:00 [info] <0.435.0> Ready to start client connection listeners
message-queue-1 2026-03-24 14:41:43.587749+00:00 [info] <0.576.0> started TCP listener on [::]:5672
message-queue-1 2026-03-24 14:41:43.587749+00:00 [info] <0.435.0> completed with 4 plugins.
message-queue-1 2026-03-24 14:41:43.660350+00:00 [info] <0.435.0> Server startup complete; 4 plugins started.
message-queue-1 2026-03-24 14:41:43.660350+00:00 [info] <0.435.0> * rabbitmq_prometheus
message-queue-1 2026-03-24 14:41:43.660350+00:00 [info] <0.435.0> * rabbitmq_management
message-queue-1 2026-03-24 14:41:43.660350+00:00 [info] <0.435.0> * rabbitmq_management_agent
message-queue-1 2026-03-24 14:41:43.660350+00:00 [info] <0.435.0> * rabbitmq_web_dispatch
message-queue-1 2026-03-24 14:41:43.816092+00:00 [info] <0.10.0> Time to start RabbitMQ: 4964 ms
message-queue-1 2026-03-24 14:41:50.072890+00:00 [info] <0.584.0> Starting AMQP connection 172.19.0.7:33906 -> 172.19.0.4:5672
message-queue-1 2026-03-24 14:41:50.080583+00:00 [info] <0.584.0> connection 172.19.0.7:33906 -> 172.19.0.4:5672: user 'guest' authenticated and granted access to vhost '/'
message-queue-1 2026-03-24 14:41:50.081420+00:00 [warning] <0.594.0> Deprecated features: 'transient_nonexcl_queues': Feature 'transient_nonexcl_queues' is deprecated.
message-queue-1 2026-03-24 14:41:50.081420+00:00 [warning] <0.594.0> By default, this feature can still be used for now.
message-queue-1 2026-03-24 14:41:50.081420+00:00 [warning] <0.594.0> Its use will not be permitted by default in a future minor RabbitMQ version and the feature will be removed fr
on a future major RabbitMQ version; actual versions to be determined.
message-queue-1 2026-03-24 14:41:50.081420+00:00 [warning] <0.594.0> To continue using this feature when it is not permitted by default, set the following parameter in your config
uration:
message-queue-1 2026-03-24 14:41:50.081420+00:00 [warning] <0.594.0> "deprecated_features.permit_transient_nonexcl_queues = true"
message-queue-1 2026-03-24 14:41:50.081420+00:00 [warning] <0.594.0> To test RabbitMQ as if the feature was removed, set this in your configuration:
message-queue-1 2026-03-24 14:41:50.081420+00:00 [warning] <0.594.0> "deprecated_features.permit_transient_nonexcl_queues = false"
```

- docker compose logs shop-db

```
llucsanchez@swarm-manager: /opt/shopmicro
shop-db-1 2026-03-24T14:41:39.605617Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
shop-db-1 2026-03-24T14:41:40.865392Z 0 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Please consider switching off the --initialize-insecure option.
shop-db-1 2026-03-24T14:41:43.575401Z 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
shop-db-1 2026-03-24 14:41:43+00:00 [Note] [Entrypoint]: Database files initialized
shop-db-1 2026-03-24 14:41:43+00:00 [Note] [Entrypoint]: Starting temporary server
shop-db-1 2026-03-24T14:41:43.618429Z 0 [System] [MY-015015] [Server] MySQL Server - start.
shop-db-1 2026-03-24T14:41:43.824278Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 9.6.0) starting as process 116
shop-db-1 2026-03-24T14:41:43.824278Z 0 [System] [MY-015590] [Server] MySQL Server has access to 2 logical CPUs.
shop-db-1 2026-03-24T14:41:43.824278Z 0 [System] [MY-015590] [Server] MySQL Server has access to 2063314944 bytes of physical memory.
shop-db-1 2026-03-24T14:41:43.845995Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
shop-db-1 2026-03-24T14:41:44.245419Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
shop-db-1 2026-03-24T14:41:44.560671Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
shop-db-1 2026-03-24T14:41:44.560699Z 0 [System] [MY-013602] [Server] Channel mysql main configured to support TLS. Encrypted connections are now supported for this channel.
shop-db-1 2026-03-24T14:41:44.564668Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consid
er choosing a different directory.
shop-db-1 2026-03-24T14:41:44.595396Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Socket: /var/run/mysqld/mysqld.sock
shop-db-1 2026-03-24T14:41:44.595432Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '9.6.0' socket: '/var/run/mysqld/mysqld.sock' port: 0 MySQL Com
munity Server - GPL.
shop-db-1 2026-03-24 14:41:44+00:00 [Note] [Entrypoint]: Temporary server started.
shop-db-1 2026-03-24T14:41:47.385134Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 9.6.0) starting as process 1
shop-db-1 2026-03-24T14:41:47.385152Z 0 [System] [MY-015590] [Server] MySQL Server has access to 2 logical CPUs.
shop-db-1 2026-03-24T14:41:47.385160Z 0 [System] [MY-015590] [Server] MySQL Server has access to 2063314944 bytes of physical memory.
shop-db-1 2026-03-24T14:41:47.401333Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
shop-db-1 2026-03-24T14:41:47.774171Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
shop-db-1 2026-03-24T14:41:48.019125Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
shop-db-1 2026-03-24T14:41:48.019160Z 0 [System] [MY-013602] [Server] Channel mysql main configured to support TLS. Encrypted connections are now supported for this channel.
shop-db-1 2026-03-24T14:41:48.023109Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consid
er choosing a different directory.
shop-db-1 2026-03-24T14:41:48.064219Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind address: '::' port: 33060, socket: /var/run/mysqld/mysqld.sock
shop-db-1 2026-03-24T14:41:48.064236Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '9.6.0' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL
Community Server - GPL.
llucsanchez@swarm-manager: /opt/shopmicro
```

- docker compose logs cache

```

llucsanchez@swarm-manager: /opt/shopmicro
cache-1 1:1M 24 Mar 2026 14:41:38.555 * <bf> ( bf-error-rate : 0.01 )
cache-1 1:1M 24 Mar 2026 14:41:38.555 * <bf> ( bf-initial-size : 100 )
cache-1 1:1M 24 Mar 2026 14:41:38.555 * <bf> ( bf-expansion-factor : 2 )
cache-1 1:1M 24 Mar 2026 14:41:38.555 * <bf> ( cf-bucket-size : 2 )
cache-1 1:1M 24 Mar 2026 14:41:38.555 * <bf> ( cf-initial-size : 1024 )
cache-1 1:1M 24 Mar 2026 14:41:38.555 * <bf> ( cf-max-iterations : 20 )
cache-1 1:1M 24 Mar 2026 14:41:38.555 * <bf> ( cf-expansion-factor : 1 )
cache-1 1:1M 24 Mar 2026 14:41:38.555 * <bf> ( cf-max-expansions : 32 )
cache-1 1:1M 24 Mar 2026 14:41:38.555 * <bf> ]
cache-1 1:1M 24 Mar 2026 14:41:38.555 * Module 'bf' loaded from /usr/local/lib/redis/modules//redisbloom.so
cache-1 1:1M 24 Mar 2026 14:41:38.583 * <search> Redis version found by RedisSearch: 8.6.1 - oss
cache-1 1:1M 24 Mar 2026 14:41:38.583 * <search> RedisSearch version 8.6.0 (Git=7782b99)
cache-1 1:1M 24 Mar 2026 14:41:38.583 * <search> Low level api version 1 initialized successfully
cache-1 1:1M 24 Mar 2026 14:41:38.584 * <search> gc: ON, prefix min length: 2, min word length to stem: 4, prefix max expansions: 200, query timeout (ms): 500, timeout policy: return, oom
policy: return, cursor read size: 1000, cursor max idle (ms): 300000, max number of search results: 1000000, default scorer: BM25STD,
cache-1 1:1M 24 Mar 2026 14:41:38.585 * <search> Initialized thread pools!
cache-1 1:1M 24 Mar 2026 14:41:38.585 * <search> Disabled workers threadpool of size 0
cache-1 1:1M 24 Mar 2026 14:41:38.588 * <search> Subscribe to config changes
cache-1 1:1M 24 Mar 2026 14:41:38.588 * <search> Subscribe to cluster slot migration events
cache-1 1:1M 24 Mar 2026 14:41:38.588 * <search> Enabled role change notification
cache-1 1:1M 24 Mar 2026 14:41:38.589 * <search> Cluster configuration: AUTO partitions, type: 0, coordinator timeout: 0ms
cache-1 1:1M 24 Mar 2026 14:41:38.593 * <timeseries> Module 'timeseries' loaded from /usr/local/lib/redis/modules//redistimeseries.so
cache-1 1:1M 24 Mar 2026 14:41:38.591 * <timeseries> RedisTimeseries version 80600, git sha=05fd355db748676861dc4c17d19c8c1ca74c0154
cache-1 1:1M 24 Mar 2026 14:41:38.591 * <timeseries> Redis version found by RedisTimeseries: 8.6.1 - oss
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> Registering configuration options: [
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> ( ts-compaction-policy :
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> ( ts-num-threads : 3 )
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> ( ts-retention-policy : 0 )
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> ( ts-duplicate-policy : block )
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> ( ts-chunk-size-bytes : 4096 )
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> ( ts-encoding : compressed )
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> ( ts-ignore-max-time-diff : 0 )
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> ( ts-ignore-max-val-diff : 0.000000 )
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> ]
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> Detected redis oss
cache-1 1:1M 24 Mar 2026 14:41:38.592 * <timeseries> Subscribe to ASM events
cache-1 1:1M 24 Mar 2026 14:41:38.593 * <timeseries> Enabled diskless replication
cache-1 1:1M 24 Mar 2026 14:41:38.593 * <timeseries> Module 'timeseries' loaded from /usr/local/lib/redis/modules//redistimeseries.so
cache-1 1:1M 24 Mar 2026 14:41:38.599 * <ReJSON> Created new data type 'ReJSON-RL'
cache-1 1:1M 24 Mar 2026 14:41:38.603 * <ReJSON> Version: 80600, git sha: unknown branch: unknown
cache-1 1:1M 24 Mar 2026 14:41:38.603 * <ReJSON> Exported RedisJSON V1 API
cache-1 1:1M 24 Mar 2026 14:41:38.603 * <ReJSON> Exported RedisJSON V2 API
cache-1 1:1M 24 Mar 2026 14:41:38.603 * <ReJSON> Exported RedisJSON V3 API
cache-1 1:1M 24 Mar 2026 14:41:38.603 * <ReJSON> Exported RedisJSON V4 API
cache-1 1:1M 24 Mar 2026 14:41:38.603 * <ReJSON> Exported RedisJSON V5 API
cache-1 1:1M 24 Mar 2026 14:41:38.603 * <ReJSON> Exported RedisJSON V6 API
cache-1 1:1M 24 Mar 2026 14:41:38.603 * <ReJSON> Enabled diskless replication
cache-1 1:1M 24 Mar 2026 14:41:38.603 * <ReJSON> Initialized shared string cache, thread safe: true.
cache-1 1:1M 24 Mar 2026 14:41:38.603 * <ReJSON> Module 'ReJSON' loaded from /usr/local/lib/redis/modules//rejson.so
cache-1 1:1M 24 Mar 2026 14:41:38.604 * <ReJSON> Acquired RedisJSON V6 API
cache-1 1:1M 24 Mar 2026 14:41:38.604 * <ReJSON> Server initialized
cache-1 1:1M 24 Mar 2026 14:41:38.605 * Ready to accept connections tcp
cache-1 1:1M 24 Mar 2026 14:41:38.605 * WARNING: Redis does not require authentication and is not protected by network restrictions. Redis will accept connections from any IP address on a
ny network interface.
llucsanchez@swarm-manager: /opt/shopmicro$

```

- docker compose logs message-queue

```

llucsanchez@swarm-manager: /opt/shopmicro
message-queue-1 1:2026-03-24 14:41:43.403821+00:00 [info] <209.0> Successfully set permissions for user 'guest' in virtual host '/' to '.*', '.*', '.*'
message-queue-1 1:2026-03-24 14:41:43.403878+00:00 [info] <209.0> Running boot step rabbit_observer_cli defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.403997+00:00 [info] <209.0> Running boot step rabbit_core_metrics_gc defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404061+00:00 [info] <209.0> Running boot step background_gc defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404120+00:00 [info] <209.0> Running boot step rabbit_observer_cli_classic_queues defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404145+00:00 [info] <209.0> Running boot step rabbit_observer_cli_classic_queues defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404159+00:00 [info] <209.0> Running boot step routing_ready defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404167+00:00 [info] <209.0> Running boot step pre_flight defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404176+00:00 [info] <209.0> Running boot step notify_cluster defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404184+00:00 [info] <209.0> Running boot step networking defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404208+00:00 [info] <209.0> Running boot step rabbit_quorum_queue_periodic_membership_reconciliation defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404307+00:00 [info] <209.0> Running boot step definition_import_worker_pool defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404324+00:00 [info] <209.0> Starting worker pool 'definition_import_pool' with 2 processes in it
message-queue-1 1:2026-03-24 14:41:43.404401+00:00 [info] <209.0> Running boot step cluster_name defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.404440+00:00 [info] <209.0> Running boot step internal_cluster_id to 'rabbitmq-cluster-id-NALmM_f-xc5pXj2fj4k72g'
message-queue-1 1:2026-03-24 14:41:43.409711+00:00 [info] <209.0> Running boot step virtual_host_reconciliation defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.409901+00:00 [info] <209.0> Running boot step direct_client defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.410021+00:00 [info] <209.0> Running boot step logger_exchange defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.412498+00:00 [info] <209.0> Running boot step cluster_tags defined by app rabbit
message-queue-1 1:2026-03-24 14:41:43.414496+00:00 [info] <209.0> Running boot step rabbit_management_load_definitions defined by app rabbitmq_management
message-queue-1 1:2026-03-24 14:41:43.414836+00:00 [info] <435.0> Resetting node maintenance status
message-queue-1 1:2026-03-24 14:41:43.529738+00:00 [warning] <457.0> Deprecated features: 'management_metrics_collection'. Feature 'management_metrics_collection' is deprecated.
message-queue-1 1:2026-03-24 14:41:43.529738+00:00 [warning] <457.0> By default, this feature can still be used for now.
message-queue-1 1:2026-03-24 14:41:43.529738+00:00 [warning] <457.0> Its use will not be permitted by default in a future minor RabbitMQ version and the feature will be removed from a fu
ture major RabbitMQ version; actual versions to be determined.
message-queue-1 1:2026-03-24 14:41:43.529738+00:00 [warning] <457.0> To continue using this feature when it is not permitted by default, set the following parameter in your configuration
:
message-queue-1 1:2026-03-24 14:41:43.529738+00:00 [warning] <457.0> "deprecated_features.permit_management_metrics_collection = true"
message-queue-1 1:2026-03-24 14:41:43.529738+00:00 [warning] <457.0> To test RabbitMQ as if the feature was removed, set this in your configuration:
message-queue-1 1:2026-03-24 14:41:43.529738+00:00 [warning] <457.0> "deprecated_features.permit_management_metrics_collection = false"
message-queue-1 1:2026-03-24 14:41:43.567955+00:00 [info] <493.0> Management plugin: HTTP (non-TLS) listener started on port 15672
message-queue-1 1:2026-03-24 14:41:43.568156+00:00 [info] <521.0> Statistics database started.
message-queue-1 1:2026-03-24 14:41:43.568216+00:00 [info] <520.0> Starting worker pool 'management_worker_pool' with 3 processes in it
message-queue-1 1:2026-03-24 14:41:43.568255+00:00 [info] <532.0> Prometheus metrics: HTTP (non-TLS) listener started on port 15692
message-queue-1 1:2026-03-24 14:41:43.568354+00:00 [info] <435.0> Ready to start client connection listeners
message-queue-1 1:2026-03-24 14:41:43.587749+00:00 [info] <576.0> started TCP listener on [:]:15672
message-queue-1 1:2026-03-24 14:41:43.587749+00:00 [info] <576.0> started TCP listener on [:]:15672
message-queue-1 1:2026-03-24 14:41:43.587749+00:00 [info] <576.0> started TCP listener on [:]:15672
message-queue-1 1:2026-03-24 14:41:43.660350+00:00 [info] <435.0> Server startup completed; 4 plugins started.
message-queue-1 1:2026-03-24 14:41:43.660350+00:00 [info] <435.0> * rabbitmq_prometheus
message-queue-1 1:2026-03-24 14:41:43.660350+00:00 [info] <435.0> * rabbitmq_management
message-queue-1 1:2026-03-24 14:41:43.660350+00:00 [info] <435.0> * rabbitmq_management_agent
message-queue-1 1:2026-03-24 14:41:43.660350+00:00 [info] <435.0> * rabbitmq_web_dispatch
message-queue-1 1:2026-03-24 14:41:43.816092+00:00 [info] <10.0> Time to start RabbitMQ: 464 ms
message-queue-1 1:2026-03-24 14:41:50.080583+00:00 [info] <584.0> accepting AMQP connection 172.19.0.7:33986 -> 172.19.0.4:5672
message-queue-1 1:2026-03-24 14:41:50.081420+00:00 [warning] <594.0> Deprecated features: 'transient_nonexcl_queues'. Feature 'transient_nonexcl_queues' is deprecated.
message-queue-1 1:2026-03-24 14:41:50.081420+00:00 [warning] <594.0> By default, this feature can still be used for now.
message-queue-1 1:2026-03-24 14:41:50.081420+00:00 [warning] <594.0> Its use will not be permitted by default in a future minor RabbitMQ version and the feature will be removed from a fu
ture major RabbitMQ version; actual versions to be determined.
message-queue-1 1:2026-03-24 14:41:50.081420+00:00 [warning] <594.0> To continue using this feature when it is not permitted by default, set the following parameter in your configuration
:
message-queue-1 1:2026-03-24 14:41:50.081420+00:00 [warning] <594.0> "deprecated_features.permit_transient_nonexcl_queues = true"
message-queue-1 1:2026-03-24 14:41:50.081420+00:00 [warning] <594.0> To test RabbitMQ as if the feature was removed, set this in your configuration:
message-queue-1 1:2026-03-24 14:41:50.081420+00:00 [warning] <594.0> "deprecated_features.permit_transient_nonexcl_queues = false"
llucsanchez@swarm-manager: /opt/shopmicro$

```

Video Demostratiu Fase 1

A continuació, hem realitzat un video demostratiu on despleguem des de 0 tota la solució que hem explicat fins ara.

Enllaços:

- [Youtube](#) 
- [Google Drive](#) 
- [GitHub](#)  (Documentació) **FALTA PONER**

Minutatge:

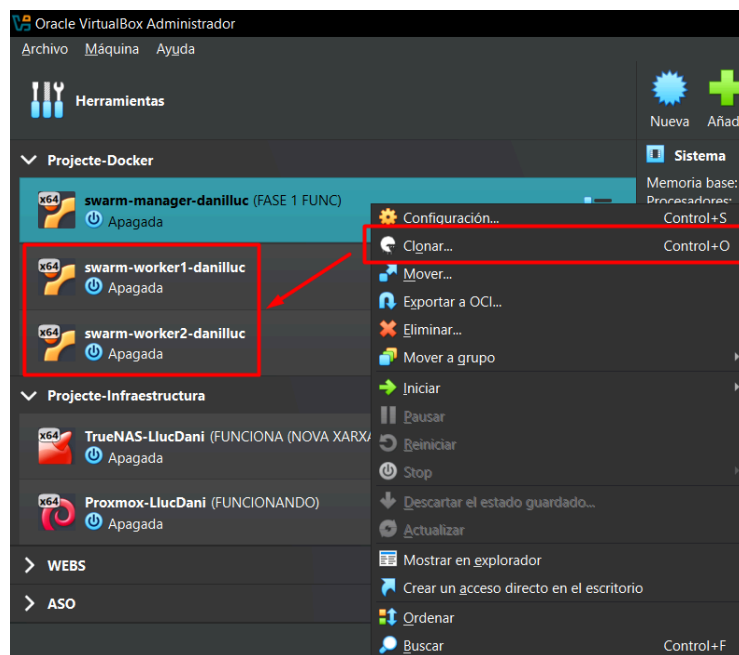
- 00:00 - 00:09 → Demostració Github del projecte
- 00:10 - 00:27 → Màquina Manager funcionant correctament + IP assignada
- 00:28 - 00:54 → Git clone del nostre propi repositori
- 00:55 - 01:44 → docker compose up + docker ps
- 01:45 - 02:41 → Accés i demostració funcionament de la web.
- 02:42 - 03:24 → docker compose logs de rabbitMQ
- 03:25 - 04:18 → Accés a MySQL des de la terminal del contenidor
- 04:19 - 04:50 → Demostració docker-compose.yml

Fase 2: docker-swarm

Configuració de Màquines Worker

Abans de començar amb aquesta fase, cal configurar les màquines virtuals que utilitzarem com a Workers dins de la nostra solució Docker, la configuració ha de ser exactament igual que com l'hem fet amb el Manager.

Com hem comentat al principi d'aquest document, cal fer una clonació de la màquina virtual Manager per obtenir la mateixa configuració, a VirtualBox podem fer Click Dret → Clonar.



Un cop tenim els 2 workers clonats, hi accedirem i realitzarem un canvi d'IP per la que pertoqui a cada interfície, les IP les podem consultar a la [taula de direccionament](#).

Configuració IP Worker 1:

```

llucsanchez@swarm-worker1: ~
llucsanchez@swarm-worker1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid lft forever preferred lft forever
    inet6 ::1/128 scope host noprefixroute
        valid lft forever preferred lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c4:5f:f3 brd ff:ff:ff:ff:ff:ff
    inet 10.93.255.128/16 metric 100 brd 10.93.255.255 scope global dynamic enp0s3
        valid lft 85900sec preferred lft 85900sec
    inet6 fe80::a00:27ff:fec4:5ff3/64 scope link
        valid lft forever preferred lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:67:34:73 brd ff:ff:ff:ff:ff:ff
    inet 10.0.50.20/24 brd 10.0.50.255 scope global enp0s8
        valid lft forever preferred lft forever
    inet6 fe80::a00:27ff:fe67:3473/64 scope link
        valid lft forever preferred lft forever
4: enp0s9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8f:73:07 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.101/24 brd 192.168.10.255 scope global enp0s9
        valid lft forever preferred lft forever
    inet6 fe80::a00:27ff:fe8f:7307/64 scope link
        valid lft forever preferred lft forever
5: enp0s10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:58:41:00 brd ff:ff:ff:ff:ff:ff
    inet 10.0.30.61/24 brd 10.0.30.255 scope global enp0s10
        valid lft forever preferred lft forever
    inet6 fe80::a00:27ff:fe58:4100/64 scope link
  
```


Configuració IP Worker 2:

```
llucsanchez@swarm-worker2: ~  
llucsanchez@swarm-worker2:~$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:c4:cb:a2 brd ff:ff:ff:ff:ff:ff  
    inet 10.93.254.184/16 metric 100 brd 10.93.255.255 scope global dynamic enp0s3  
        valid_lft 86004sec preferred_lft 86004sec  
    inet6 fe80::a00:27ff:fec4:cba2/64 scope link  
        valid_lft forever preferred_lft forever  
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:58:34:3d brd ff:ff:ff:ff:ff:ff  
    inet 10.0.50.30/24 brd 10.0.50.255 scope global enp0s8  
        valid_lft forever preferred_lft forever  
    inet6 fe80::a00:27ff:fe58:343d/64 scope link  
        valid_lft forever preferred_lft forever  
4: enp0s9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:60:e6:5f brd ff:ff:ff:ff:ff:ff  
    inet 192.168.10.102/24 brd 192.168.10.255 scope global enp0s9  
        valid_lft forever preferred_lft forever  
    inet6 fe80::a00:27ff:fe60:e65f/64 scope link  
        valid_lft forever preferred_lft forever  
5: enp0s10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:2f:d2:11 brd ff:ff:ff:ff:ff:ff  
    inet 10.0.30.62/24 brd 10.0.30.255 scope global enp0s10  
        valid_lft forever preferred_lft forever  
    inet6 fe80::a00:27ff:fe2f:d211/64 scope link  
        valid_lft forever preferred_lft forever
```

Un cop ja els tenim a les xarxes que pertoquen i es poden comunicar entre ells, ja podem començar amb la primera part d'aquesta fase. Cal recordar que al haver fet una clonació directament del Manager, ja disposem de tots els fitxers de Docker necessaris instal·lats a les màquines.

Unió de Workers dins del Swarm

Per unir les màquines dins del Swarm, és tan senzill com accedir al Manager i introduir la comanda “docker swarm init --advertise-addr {IP_MANAGER}”. Cal posar la IP de la xarxa interna dedicada per Docker que hem definit a la fase 1.

```
llucsanchez@swarm-manager: ~
llucsanchez@swarm-manager:~$ docker swarm init --advertise-addr 10.0.50.10
Swarm initialized: current node (ymf6nkcykl2a9xa2gymvzkc0x) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3ky6eaozq7a43xr2rmkvxmm43j3l065k3f4laskrbqzrff5htr-6pphq57hbyh3ldty10h5fu72u 10.0.50.10:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
llucsanchez@swarm-manager:~$
```

Per la terminal ens retornen una comanda que serveix per poder unir als workers utilitzant un token que hem generat al Manager, aquesta comanda cal copiar-la i introduir-la als 2 Workers restants.

Worker1:

```
llucsanchez@swarm-worker1: ~
llucsanchez@swarm-worker1:~$ ip a | grep enp0s8
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet 10.0.50.20/24 brd 10.0.50.255 scope global enp0s8
llucsanchez@swarm-worker1:~$ hostname
swarm-worker1
llucsanchez@swarm-worker1:~$ docker swarm join --token SWMTKN-1-3ky6eaozq7a43xr2rmkvxmm43j3l065k3f4laskrbqzrff5htr-6pphq57hbyh3ldty10h5fu72u 10.0.50.10:2377
This node joined a swarm as a worker.
llucsanchez@swarm-worker1:~$
```

Worker 2:

```
llucsanchez@swarm-worker2: ~
llucsanchez@swarm-worker2:~$ ip a | grep enp0s8
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet 10.0.50.30/24 brd 10.0.50.255 scope global enp0s8
llucsanchez@swarm-worker2:~$ hostname
swarm-worker2
llucsanchez@swarm-worker2:~$ docker swarm join --token SWMTKN-1-3ky6eaozq7a43xr2rmkvxmm43j3l065k3f4laskrbqzrff5htr-6pphq57hbyh3ldty10h5fu72u 10.0.50.10:2377
This node joined a swarm as a worker.
llucsanchez@swarm-worker2:~$
```

Un cop ja hem realitzat el procés, amb la comanda “docker node ls” al manager, podem comprovar si les màquines es troben dins del Swarm, ens ho indica amb un **Status: Ready**.

```
llucsanchez@swarm-manager: ~
llucsanchez@swarm-manager:~$ docker node ls
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS  ENGINE VERSION
ymf6nkcykl2a9xa2gymvzkc0x * swarm-manager   Ready     Active           Leader           29.3.0
c7ddelrx1xa0q0legop0jq9d6 swarm-worker1   Ready     Active           29.3.0
hzccfkr5ubpfng7tps028tlwg swarm-worker2   Ready     Active           29.3.0
llucsanchez@swarm-manager:~$
```

El *Leader* representa la màquina Manager, aquesta s'encarrega d'anar distribuint els serveis als altres nodes segons els requeriments del moment, si es para, s'atura el swarm.

Adaptació docker-stack

- Penjar imatges a DockerHub

Abans de començar amb l'adaptació del fitxer docker-compose, pujarem les imatges que tenim actualment a Docker Hub, aquest actua com un GitHub personal on podem anar guardant i versionant les imatges segons les nostres necessitats, d'aquesta forma podem treballar amb les imatges de forma centralitzada i per internet.

Dins del manager, amb la comanda “docker login”, podem iniciar sessió al servei de Docker des de la terminal, cal seguir les instruccions i acceptar l'autorització al nostre navegador:

```
llucsanchez@swarm-manager: /opt/shopmicro
llucsanchez@swarm-manager:/opt/shopmicro$ docker login

USING WEB-BASED LOGIN

Info → To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: SPDQ-LKWS
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate
Waiting for authentication in the browser...
```

Un cop hem acceptat l'autorització:

```
llucsanchez@swarm-manager: /opt/shopmicro
llucsanchez@swarm-manager:/opt/shopmicro$ docker login

USING WEB-BASED LOGIN

Info → To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: SPDQ-LKWS
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate
Waiting for authentication in the browser...

WARNING! Your credentials are stored unencrypted in '/home/llucsanchez/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
llucsanchez@swarm-manager:/opt/shopmicro$
```

Tot seguit, amb la comanda “docker images” podem consultar les imatges Docker que tenim actualment al node Manager, aquestes imatges s’han generat durant la Fase 1, la idea es poder guardar-les a un repositori DockerHub.

```
llucsanchez@swarm-manager: /opt/shopmicro
llucsanchez@swarm-manager:/opt/shopmicro$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
mysql:9.6	24e450bbd24f	1.27GB	283MB	U
rabbitmq:4.2.5-management-alpine	b618738ea52c	272MB	88.7MB	U
redis:8.6.1-alpine	2afba59292f2	134MB	34.9MB	U
shopmicro-api-gateway:latest	ddd5c8e62af5	100MB	29.4MB	U
shopmicro-frontend:latest	5ebc353602bc	707MB	176MB	U
shopmicro-notification-service:latest	54e536802e15	89.2MB	21.8MB	U
shopmicro-order-service:latest	9981a84f34e4	140MB	37.1MB	U
shopmicro-product-service:latest	402cbf7721c6	137MB	36.5MB	U
shopmicro-user-service:latest	8b6f37433bbf	130MB	34.9MB	U

```
llucsanchez@swarm-manager:/opt/shopmicro$
```

Ara, per cada imatge cal que li indiquem un “tag”, aquest l'utilitzarem per indicar la versió de la imatge que estem utilitzant, en el nostre cas “v1”:

```
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-frontend lsanchez5420/shopmicro-frontend:v1
[sudo] password for llucsanchez:
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-api-gateway lsanchez5420/shopmicro-api-gateway:v1
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-product-service lsanchez5420/shopmicro-product-service:v1
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-order-service lsanchez5420/shopmicro-order-service:v1
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-user-service lsanchez5420/shopmicro-user-service:v1
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-notification-service lsanchez5420/shopmicro-notification-service:v1
llucsanchez@swarm-manager: /opt/shopmicro$
```

Totes les imatges amb Tag:

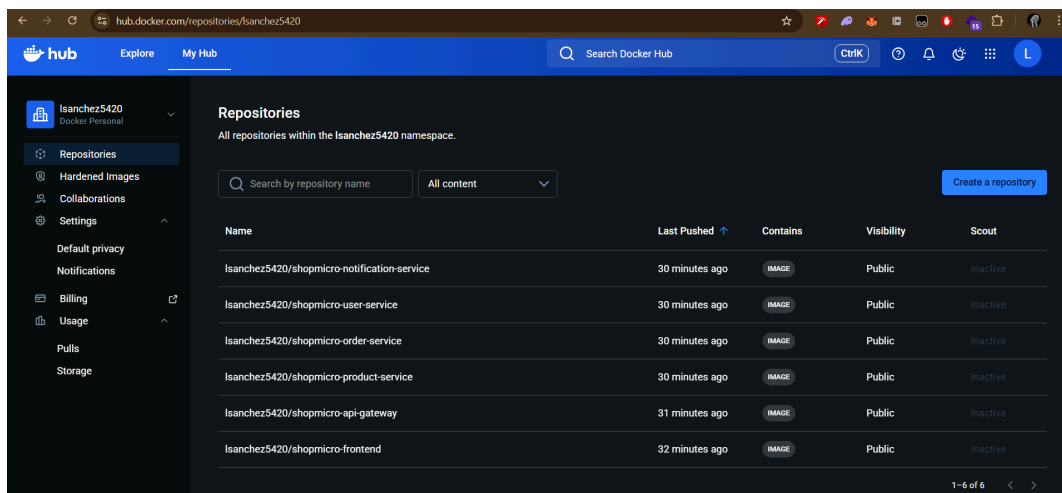
```
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-frontend lsanchez5420/shopmicro-frontend:v1
[sudo] password for llucsanchez:
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-api-gateway lsanchez5420/shopmicro-api-gateway:v1
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-product-service lsanchez5420/shopmicro-product-service:v1
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-order-service lsanchez5420/shopmicro-order-service:v1
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-user-service lsanchez5420/shopmicro-user-service:v1
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker tag shopmicro-notification-service lsanchez5420/shopmicro-notification-service:v1
llucsanchez@swarm-manager: /opt/shopmicro$ docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
lsanchez5420/shopmicro-api-gateway:v1	ddd5c9e62af5	100MB	29.4MB	U
lsanchez5420/shopmicro-frontend:v1	5ebc353602bc	707MB	176MB	U
lsanchez5420/shopmicro-notification-service:v1	54e536802e15	89.2MB	21.8MB	U
lsanchez5420/shopmicro-order-service:v1	9981a64f34e4	140MB	37.1MB	U
lsanchez5420/shopmicro-product-service:v1	402c8f7721c6	137MB	36.5MB	U
lsanchez5420/shopmicro-user-service:v1	8b6ef37433bf	130MB	34.9MB	U

Per finalitzar, amb “docker push” podem pujar les imatges al nostre DockerHub (igual que com ho fem a GitHub), això **s’ha de fer per cada imatge que vulguem guardar**.

```
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker push lsanchez5420/shopmicro-frontend:v1
The push refers to repository [docker.io/lsanchez5420/shopmicro-frontend]
a0a80a1734a9: Pushed
1f02717bf2c3: Pushed
f9dd300160c5: Pushed
01e779a69be5: Pushed
8659816aaf55: Pushed
1513128de1f2: Pushed
4f4fb700ef54: Mounted from library/redis
4302931ae550: Pushed
7bf983cfeb1f1: Pushed
de62b766946a: Pushed
f69c0ef6aba1: Pushed
c6756d23f76c: Pushed
64cef85a82b1: Pushed
623055e7ebef: Pushed
3334ef6307d6: Pushed
ec781dee3f47: Pushed
0cd6ecab7e20: Pushed
a22721440bff: Pushed
v1: digest: sha256:5ebc353602bc5db55be47b4ee146b3140d0039853286d3e2ce38e2ce1a4804e5 size: 856
llucsanchez@swarm-manager: /opt/shopmicro$
```

Resultat final a Docker Hub:



- Fitxer docker-stack

Ara cal copiar l'arxiu **docker-compose.yml** de la fase 1 i **modificar-lo** fins adaptar-ho a docker-stack, simplement cal fer algunes modificacions segons els requeriments.

*** Per no haver de posar moltes captures de l'arxiu en aquest document, adjuntem un enllaç a GitHub on es pot visualitzar tot l'arxiu.*

[Enllaç](#) 

Un cop ja hem fet la còpia del docker stack, podem indicar el fitxer docker-compose.yml com a old fent un canvi de nom, aquest en principi no l'utilitzarem per el moment.

```
llucsanchez@swarm-manager: /opt/shopmicro
llucsanchez@swarm-manager:~$ cd /opt/shopmicro
llucsanchez@swarm-manager:/opt/shopmicro$ ls -l
total 52
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 16 18:54 api-gateway
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 23 15:37 db-init
-rw-rw-r-- 1 llucsanchez llucsanchez 3104 Mar 23 19:02 docker-compose.yml.old
-rw-rw-r-- 1 llucsanchez llucsanchez 4778 Mar 27 17:45 docker-stack.yml
drwxrwxr-x 3 llucsanchez llucsanchez 4096 Mar 16 18:34 frontend
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 16 18:19 frontend_old
drwxr-xr-x 3 root root 4096 Mar 23 19:02 logs
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 24 15:49 logs temp
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 16 16:26 notification-service
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 16 16:26 order-service
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 16 16:26 product-service
drwxrwxr-x 2 llucsanchez llucsanchez 4096 Mar 16 16:26 user-service
llucsanchez@swarm-manager:/opt/shopmicro$
```

Ara si, encara que els 2 arxius son molt similars, hi ha alguns canvis importants que cal fer perquè funcioni correctament el desplegament del swarm:

- Migració de Ports a la sintaxi llarga.

```
api-gateway:
  image: lsanchez5420/shopmicro-api-gateway:v1
  ports:
    - target: 80
      published: 80
      protocol: tcp
      mode: ingress
    - target: 443
      published: 443
      protocol: tcp
      mode: ingress
  networks:
    - frontend-net
    - api-net
  volumes:
    - nginx_logs:/var/log/nginx
  deploy:
    replicas: 2
    update_config:
      parallelism: 1
      delay: 10s
    restart_policy:
      condition: on-failure
```

Aquesta sintaxi ens permet **evitar errors i personalitzar** la forma en la que volem que es connectin els nodes als ports, per exemple, el mode "ingress" indica que obre el port 80 a **tots els nodes del cluster alhora**, si l'usuari introdueix la IP del worker-1 al navegador però el servei s'està executant en un altre node, el Swarm reenvia la petició a l'altre worker de forma transparent, així **la botiga sempre pot respondre independentment del node**.

- Paràmetre “deploy”

```
api-gateway:
  image: lsanchez5420/shopmicro-api-gateway:v1
  ports:
    - target: 80
      published: 80
      protocol: tcp
      mode: ingress
    - target: 443
      published: 443
      protocol: tcp
      mode: ingress
  networks:
    - frontend-net
    - api-net
  volumes:
    - nginx_logs:/var/log/nginx
  deploy:
    replicas: 2
    update_config:
      parallelism: 1
      delay: 10s
    restart_policy:
      condition: on-failure
```

Aquest paràmetre s'utilitza exclusivament a Docker Swarm, és el **cervell de l'orquestració**, l'hem configurat segons els requisits de la Fase:

- **replicas: 2** → L'orquestrador sempre ha de mantenir 2 contenidors actius del servei repartits entre diferents nodes per garantir Alta Disponibilitat.
- **update_config** → Configuració que serveix per fer actualitzacions del codi sense cap tall del servei
- **parallelism: 1** → Si actualitzem una versió de la web, Docker apagarà i actualitzarà els contenidors **d'un en un**.
- **delay: 10s** → Obliga a Docker a esperar 10 segons entre actualitzacions per comprovar que el nou contenidor es troba en estat *Healthy* abans d'apagar el següent.
- **restart-policy** → Si algun servei retorna un error a l'arrancar, el contenidor es tanca i Swarm el recrea automàticament i torna a provar d'engegar-lo.

```
retries: 3
deploy:
  placement:
    constraints: [node.role == manager]
  restart_policy:
    condition: on-failure
```

- **constraints** → Serveix per indicar que el servei només s'executa al node manager, això ho fem pels 3 serveis principals de la nostra solució (*MySQL*, *Redis*, *RabbitMQ*)

- Xarxes Overlay

```
networks:
  frontend-net:
    driver: overlay
  api-net:
    driver: overlay
  backend-net:
    driver: overlay
```

Hem modificat les xarxes existents que teníem a mode “overlay”, aquest s'utilitza a Swarm perquè els serveis es puguin **comunicar entre tots els nodes de la xarxa**, uneix el manager amb els workers, per exemple, si la BBDD es troba a un node i el frontend a un d'altre, aquests es poden comunicar sense problema.

A més a més, hem afegit una xarxa extra per millorar la seguretat de la solució “api-net”, amb això aconseguim dividir per complet el Frontend amb el Backend, perquè aquests es comuniquin han de passar per la xarxa API obligatòriament.

- Volumes NFS

```
volumes:
  shop_db_data:
    driver: local
    driver_opts:
      type: nfs
      o: "addr=10.0.30.10,nfsvers=4,rw"
      device: ":/mnt/pool-principal/docker-volumes/shop_db_data"
  rabbitmq_data:
    driver: local
    driver_opts:
      type: nfs
      o: "addr=10.0.30.10,nfsvers=4,rw"
      device: ":/mnt/pool-principal/docker-volumes/rabbitmq_data"
  nginx_logs:
    driver: local
    driver_opts:
      type: nfs
      o: "addr=10.0.30.10,nfsvers=4,rw"
      device: ":/mnt/pool-principal/docker-volumes/nginx_logs"
```

Hem decidit guardar els volums de Docker a un TrueNAS mitjançant [NFS](#), per això cal utilitzar un driver de tipus NFS dins de l'arxiu.

Aquesta part l'explicarem més detallada al següent apartat, on configurarem el TrueNAS per rebre aquestes dades.

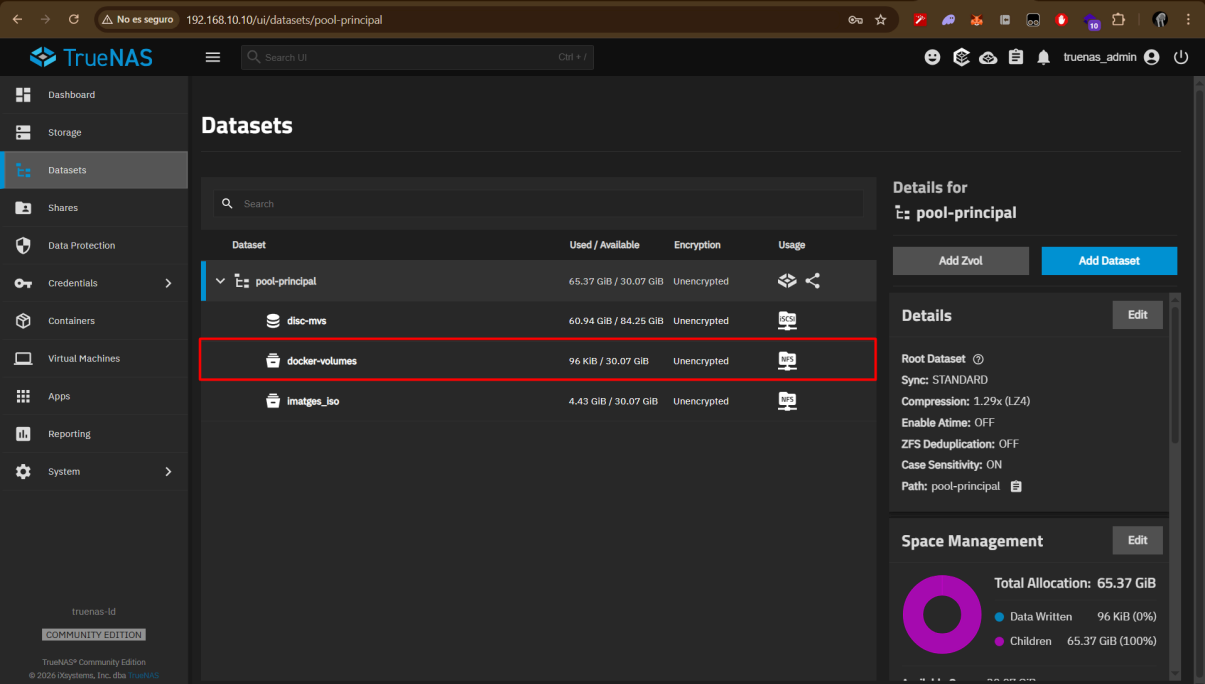
Configuració TrueNAS amb NFS

L'objectiu d'aquesta part és poder guardar els volums de Docker en un servidor de forma centralitzada, així en cas de que alguna màquina deixi de funcionar, tenim tota la informació segura en un altre servidor extern, la potència d'això és poder gestionar backups/rèpliques d'aquests volums per no perdre informació important (sobretot de la BBDD).

En el nostre cas, aprofitarem el TrueNAS que vam muntar al mini projecte de [Virtualització](#). Per poder comunicar-nos amb aquest servidor, utilitzarem l'altra xarxa interna que hem definit als workers des del principi 10.0.30.0/24.

**** Per qualsevol informació extra sobre el TrueNAS, es pot consultar l'[Annex corresponent](#).**

Primer de tot, cal crear un nou dataset dins del TrueNAS, aquest l'anomenarem "docker-volumes", la funció d'aquest és tindre un espai de disc on guardar les dades que necessitem.



Dataset	Used / Available	Encryption	Usage
pool-principal	65.37 GiB / 30.07 GiB	Unencrypted	
disc-mvs	60.94 GiB / 84.25 GiB	Unencrypted	
docker-volumes	96 KiB / 30.07 GiB	Unencrypted	
imatges_iso	4.43 GiB / 30.07 GiB	Unencrypted	

Details for pool-principal

Details

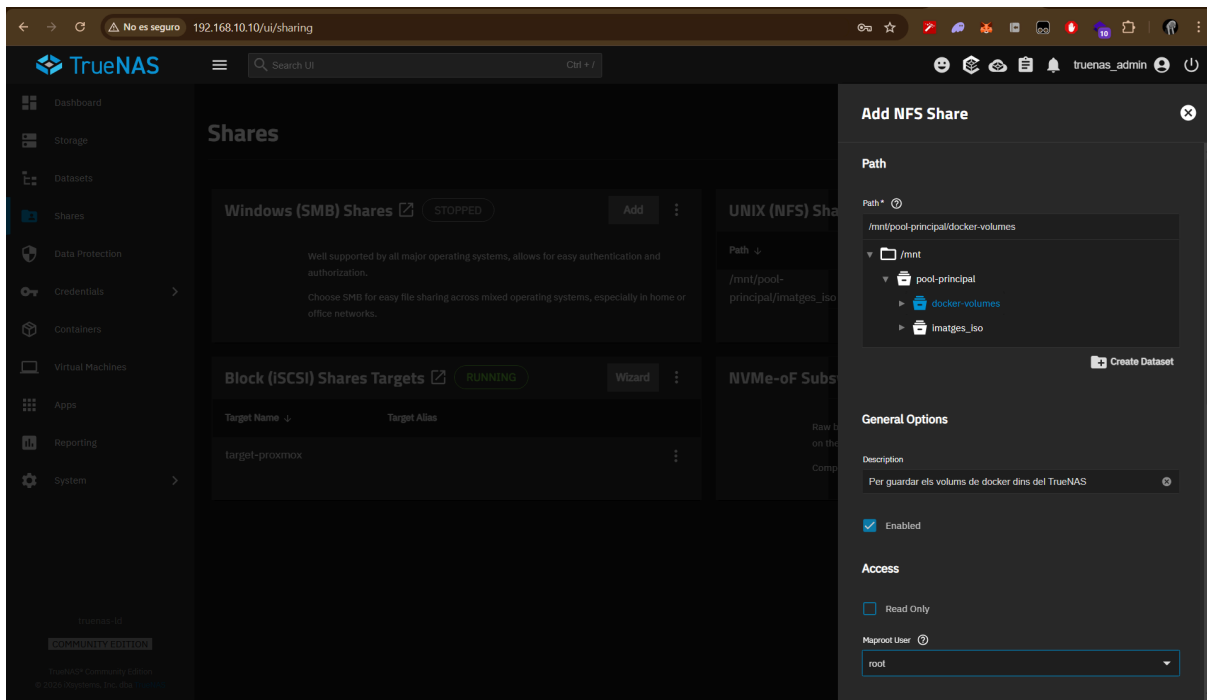
Root Dataset ⓘ
Sync: STANDARD
Compression: 1.29x (LZ4)
Enable Atime: OFF
ZFS Deduplication: OFF
Case Sensitivity: ON
Path: pool-principal

Space Management

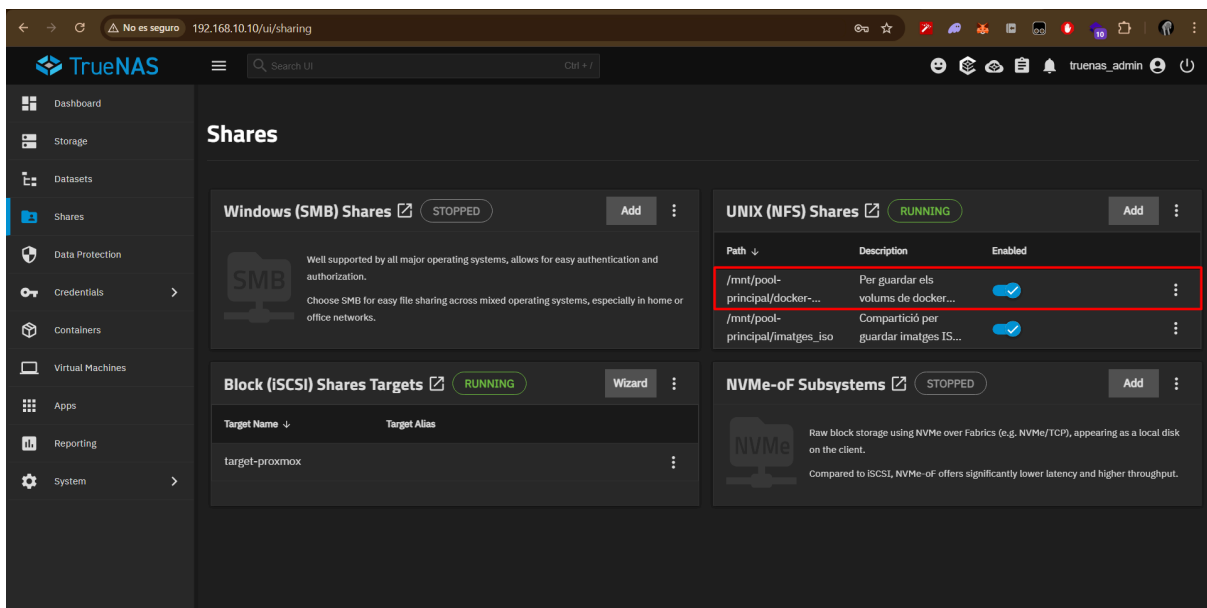
Total Allocation: 65.37 GiB

- Data Written: 96 KiB (0%)
- Children: 65.37 GiB (100%)

Tot seguit, crearem una nova estància NFS Share (Carpeta Compartida) apuntant al dataset que hem creat anteriorment, sobretot cal marcar la casella “Enabled” per activar la carpeta.



Un cop l'hem creat, s'ha de mostrar de la següent forma al nostre panell, ens indica la ruta que hem d'utilitzar per introduir les dades corresponents des de Docker.



Un cop ja tenim el TrueNAS configurat, cal instal·lar el paquet **nfs-common** a totes les màquines Ubuntu Server que estem utilitzant, així ens podem comunicar correctament amb aquest servei des de Docker.

Ubuntu Manager

 llucsanchez@swarm-manager: ~

```
llucsanchez@swarm-manager:~$ sudo apt update && sudo apt install nfs-common -y
Hit:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 https://download.docker.com/linux/ubuntu noble InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
51 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nfs-common is already the newest version (1:2.6.4-3ubuntu5.1).
0 upgraded, 0 newly installed, 0 to remove and 51 not upgraded.
llucsanchez@swarm-manager:~$
```

Ubuntu Worker 1

 llucsanchez@swarm-worker1: ~

```
llucsanchez@swarm-worker1:~$ sudo apt update && sudo apt install nfs-common -y
Hit:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 https://download.docker.com/linux/ubuntu noble InRelease
Hit:4 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
51 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nfs-common is already the newest version (1:2.6.4-3ubuntu5.1).
0 upgraded, 0 newly installed, 0 to remove and 51 not upgraded.
llucsanchez@swarm-worker1:~$
```

Ubuntu Worker 2

 llucsanchez@swarm-worker2: ~

```
llucsanchez@swarm-worker2:~$ sudo apt update && sudo apt install nfs-common -y
Hit:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 https://download.docker.com/linux/ubuntu noble InRelease
Hit:3 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
51 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nfs-common is already the newest version (1:2.6.4-3ubuntu5.1).
0 upgraded, 0 newly installed, 0 to remove and 51 not upgraded.
llucsanchez@swarm-worker2:~$
```

Per últim, només cal indicar a l'arxiu docker-stack.yml la configuració que volem utilitzar per la comunicació amb el servei NFS que acabem d'implementar:

```
volumes:
  shop_db_data:
    driver: local
    driver_opts:
      type: nfs
      o: "addr=10.0.30.10,nfsvers=4,rw"
      device: ":/mnt/pool-principal/docker-volumes/shop_db_data"
  rabbitmq_data:
    driver: local
    driver_opts:
      type: nfs
      o: "addr=10.0.30.10,nfsvers=4,rw"
      device: ":/mnt/pool-principal/docker-volumes/rabbitmq_data"
  nginx_logs:
    driver: local
    driver_opts:
      type: nfs
      o: "addr=10.0.30.10,nfsvers=4,rw"
      device: ":/mnt/pool-principal/docker-volumes/nginx_logs"
```

En el nostre cas definim el **tipus NFS**, indiquem l'adreça IP del nostre TrueNAS, la versió de NFS que volem utilitzar (*4 per evitar conflictes de compatibilitat*) i el permís de lectura-escriptura en el servei, també cal indicar la ruta que ens apareixia en el panell TrueNAS on es guardaran els arxius, com es veu a la imatge, cada volum disposa d'una carpeta compartida diferent.

Desplegament Swarm

Ja ho tenim tot llest per el primer desplegament de la nostra web en un cluster Swarm, per arrencar-lo, al manager ens dirigim a la carpeta arrel on tenim el fitxer docker-stack i utilitzarem la comadna “docker stack deploy --with-registry-auth -c docker-stack.yml shopmicro”.

El paràmetre --with-registry-auth serveix per traspasar les credencials docker login entre els nodes, així no hem de fer login a tots els nodes alhora.

```
llucsanchez@swarm-manager: /opt/shopmicro
```

```
llucsanchez@swarm-manager:/opt/shopmicro$ docker stack deploy --with-registry-auth -c docker-stack.yml shopmicro
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network shopmicro_frontend-net
Creating network shopmicro_backend-net
Creating service shopmicro_shop-db
Creating service shopmicro_user-service
Creating service shopmicro_product-service
Creating service shopmicro_cache
Creating service shopmicro_message-queue
Creating service shopmicro_order-service
Creating service shopmicro_notification-service
Creating service shopmicro_frontend
Creating service shopmicro_api-gateway
llucsanchez@swarm-manager:/opt/shopmicro$
```

Si ens dona error amb les variables d'entorn, hi han diverses formes de carregar-les en en memòria dins del servidor, en el nostre cas hem utilitzat la següent:

També es pot utilitzar: set -a; source .env; set +a

```
llucsanchez@swarm-manager: /opt/shopmicro
```

```
llucsanchez@swarm-manager:/opt/shopmicro$ export $(cat .env) && docker stack deploy -c docker-stack.yml shopmicro
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network shopmicro_backend-net
Creating network shopmicro_frontend-net
Creating service shopmicro_cache
Creating service shopmicro_product-service
Creating service shopmicro_shop-db
Creating service shopmicro_message-queue
Creating service shopmicro_notification-service
Creating service shopmicro_frontend
Creating service shopmicro_order-service
Creating service shopmicro_api-gateway
Creating service shopmicro_user-service
llucsanchez@swarm-manager:/opt/shopmicro$
```

Si no ens ha tornat cap error, podem consultar l'estat del clúster amb les següents comandes:

- Sortida de “docker stack services shopmicro”

```
llucsanchez@swarm-manager:/opt/shopmicro
```

```
llucsanchez@swarm-manager:/opt/shopmicro$ docker stack services shopmicro
ID                NAME                MODE                REPLICAS    IMAGE                                  PORTS
0i8rbg3b7im5     shopmicro_api-gateway replicated          2/2         lsanchez5420/shopmicro-api-gateway:v1 *:80->80/tcp, *:443->443/tcp
9ngbvic4ey2o     shopmicro_cache      replicated          1/1         redis:8.6.1-alpine
l5egsfswr0aq     shopmicro_frontend    replicated          2/2         lsanchez5420/shopmicro-frontend:v1
g49kib6fgk08     shopmicro_message-queue replicated          1/1         rabbitmq:4.2.5-management-alpine
n87gtxcit326     shopmicro_notification-service replicated          1/1         lsanchez5420/shopmicro-notification-service:v1 *:15672->15672/tcp
8a3ipic12a35     shopmicro_order-service replicated          2/2         lsanchez5420/shopmicro-order-service:v1
sg4lf22r1zmp     shopmicro_product-service replicated          2/2         lsanchez5420/shopmicro-product-service:v1
nix7qnfa8gmj     shopmicro_shop-db     replicated          1/1         mysql:9.6
slcmogjaewml     shopmicro_user-service replicated          2/2         lsanchez5420/shopmicro-user-service:v1
llucsanchez@swarm-manager:/opt/shopmicro$
```

Ens mostra que totes les rèpliques funcionen i s'han aixecat correctament, també ens indica els ports que s'han obert entre els nodes.

- Sortida de “docker stack ps shopmicro” (distribució als nodes)

```
llucsanchez@swarm-manager:/opt/shopmicro$ docker stack ps shopmicro
ID                NAME                IMAGE                NODE                DESIRED STATE    CURRENT STATE    ERROR    PORTS
vfvigj9yh3a8     shopmicro_api-gateway.1 lsanchez5420/shopmicro-api-gateway:v1 swarm-worker2     Running          Running about a minute ago
qyjf86bi57yz     shopmicro_api-gateway.2 lsanchez5420/shopmicro-api-gateway:v1 swarm-worker1     Running          Running about a minute ago
s8hczbwdd6qu     shopmicro_cache.1     redis:8.6.1-alpine  swarm-manager     Running          Running about a minute ago
ya5p6wchlnp4     shopmicro_frontend.1   lsanchez5420/shopmicro-frontend:v1   swarm-worker2     Running          Running about a minute ago
6x0x175myxet     shopmicro_frontend.2   lsanchez5420/shopmicro-frontend:v1   swarm-worker1     Running          Running about a minute ago
nws5j3dcscan     shopmicro_message-queue.1 rabbitmq:4.2.5-management-alpine  swarm-manager     Running          Running about a minute ago
6wir704bzeel     shopmicro_notification-service.1 lsanchez5420/shopmicro-notification-service:v1 swarm-worker1     Running          Running about a minute ago
h4gsaalbcb3x     shopmicro_order-service.1 lsanchez5420/shopmicro-order-service:v2 swarm-worker2     Running          Running about a minute ago
cz24rk02h1dp     shopmicro_order-service.2 lsanchez5420/shopmicro-order-service:v2 swarm-worker1     Running          Running about a minute ago
x08k9c4wthy      shopmicro_product-service.1 lsanchez5420/shopmicro-product-service:v2 swarm-manager     Running          Running about a minute ago
ncbvlyxxunu      shopmicro_product-service.2 lsanchez5420/shopmicro-product-service:v2 swarm-worker2     Running          Running about a minute ago
e9lhbq2fptiz     shopmicro_shop-db.1     mysql:9.6            swarm-manager     Running          Running about a minute ago
nlrnsltu7n7e     shopmicro_user-service.1 lsanchez5420/shopmicro-user-service:v2 swarm-worker2     Running          Running about a minute ago
wglef6wihda9     shopmicro_user-service.2 lsanchez5420/shopmicro-user-service:v2 swarm-worker1     Running          Running about a minute ago
llucsanchez@swarm-manager:/opt/shopmicro$
```

Tot es troba en estat “running” sense retornar cap error, significat que la nostra solució s’ha desplegat correctament.

A vegades pot passar que un contenidor falla amb l'ordre, per exemple, l'api-gateway s'ha d'executar l'últim perquè no retorni cap error, per controlar això, nosaltres ja hem indicat al docker-stack.yml que es reiniciï constantment, pel que he [llegit a internet](#), amb Docker Stack no hi ha una forma fàcil de controlar l'ordre dels contenidors com a Docker Compose.

Video Demostratiu Fase 2

A continuació adjuntem un video demostratiu on posem a prova tot el que hem implementat a la Fase 2:

- Posada en marxa docker stack des de 0 (GitHub).
- Parada en calent d'un worker.
- Posada en marxa worker de nou.
- Escalat a 4 rèpliques en un contenidor.
- Funcionament correcte de la web

Enllaços:

- [Youtube](#) 
- [Google Drive](#) 
- [GitHub](#)  (Documentació) **FALTA PONER**

Minutatge:

- 00:00 - 00:27 → GitHub del projecte + estat actual màquines a VirtualBox.
- 00:28 - 00:42 → Nodes actius dins del cluster (docker node ls).
- 00:43 - 01:12 → Clonació del repositori.
- 01:13 - 01:33 → Copiar arxiu .env + carregar variables d'entorn en memòria.
- 01:34 - 02:42 → Posada en marxa docker stack + demostració funcionament.
- 02:43 - 03:20 → Parada del servei docker al node worker2.
- 03:21 - 03:55 → Demostració node inactiu + redistribució de serveis al manager.
- 03:56 - 04:27 → Web funcionant correctament.
- 04:28 - 04:51 → Engregar servei docker al worker2.
- 04:52 - 05:23 → Demostració node actiu + estat dels serveis al manager.
- 05:24 - 06:50 → Augment 4 rèpliques al contenidor product-service + estat serveis.
- 06:51 - 07:30 → Correcte funcionament de la web.

Fase 3: Seguretat a Docker Swarm

Anàlisi de vulnerabilitats

Per comprovar si tenim vulnerabilitats dins de la nostra solució, analitzarem el fitxer docker-stack que tenim actualment.

Adjuntem una versió antiga de l'arxiu docker-stack.yml, on es poden comprovar les variables d'entorn importants posades dins del codi i guardades a l'arxiu .env, des del principi que ho hem fet d'aquesta manera.

[Enllaç](#) 

*****IMPORTANT:** No tindre en compte aquest arxiu, no està actualitzat, simplement l'utilitzem per veure la forma antiga de guardar les variables d'entorn.*

Hem detectat **una vulnerabilitat crítica d'exposició d'informació**. Tot i haver extret les contrasenyes a un fitxer .env, la instrucció environment del docker-stack.yml injecta aquestes variables com a text pla dins del contenidor.

Això permet a qualsevol actor maliciós amb accés a la consola d'un node, o mitjançant la comanda `docker service inspect`, llegir les contrasenyes de la base de dades (`DB_PASSWORD` i `DB_ROOT_PASSWORD`) en clar. Per mitigar-ho, s'ha dissenyat una migració a Docker Secrets.

Una altra que hem detectat és **l'exposició innecessària del port 15672** de RabbitMQ, així evitem forats de seguretat dins de la xarxa backend-net, es pot veure la nova versió de l'arxiu amb aquesta millora implementada en el [nostre GitHub](#).

Migració a Docker Secrets

Docker Secrets és una **eina nativa de Docker Swarm** dissenyada exclusivament per guardar i gestionar informació confidencial (com contrasenyes) de manera segura dins de l'ecosistema de Docker, d'aquesta manera no s'han d'utilitzar eines externes per emmagatzemar aquesta informació

Com hem comentat abans, una de les vulnerabilitats crítiques és que amb el fitxer `.env`, qualsevol persona que introdueixi la comanda “`docker inspect {contenedor}`” pot llegir el `docker-stack.yml` amb les credencials en text pla, per això tot l'important s'ha de migrar a Docker Secrets.

Per migrar les credencials que tenim actualment a Docker Secrets de la forma més senzilla, cal que posem les següents comandes:

 llucsanchez@swarm-manager: /opt/shopmicro

```
llucsanchez@swarm-manager:/opt/shopmicro$ printf "P@ssword_root" | docker secret create db_root_password -
ouzpt9j3ugx4z8zlylw90ikku
llucsanchez@swarm-manager:/opt/shopmicro$ printf "P@ssw0rd" | docker secret create db_password -
uqrrjn27uypm0izafqvwemdo9
llucsanchez@swarm-manager:/opt/shopmicro$
```

Un cop les tenim, podem verificar que s'han inserit correctament amb la comanda “`docker secret ls`”:

 llucsanchez@swarm-manager: /opt/shopmicro

```
llucsanchez@swarm-manager:/opt/shopmicro$ docker secret ls
ID                                NAME                DRIVER      CREATED      UPDATED
uqrrjn27uypm0izafqvwemdo9       db_password          local       2 weeks ago  2 weeks ago
ouzpt9j3ugx4z8zlylw90ikku       db_root_password     local       2 weeks ago  2 weeks ago
llucsanchez@swarm-manager:/opt/shopmicro$
```

Ara, el més important es adaptar els fitxers de codi que tenim actualment a la nova forma de guardar les credencials, com per exemple els arxius de Python que es connecten amb la BBDD:

```
# si existeix l'arxiu xifrat de Docker Secrets
if os.path.exists('/run/secrets/db_password'):
    with open('/run/secrets/db_password', 'r') as secret_file:
        DB_PASSWORD = secret_file.read().strip()
else:
    # Si no estem a Swarm, usem la variable d'entorn .env
    DB_PASSWORD = os.environ.get('DB_PASSWORD')
```

Arxiu: *product-service/app.py*

Cal fer aquesta modificació a tots els arxius Python de la nostra aplicació per a que pugui processar les credencials correctament.

Quan ja tenim tots els arxius modificats, cal pujar una nova versió de cada imatge dins del nostre Docker Hub, cal seguir els [mateixos passos](#) que hem fet anteriorment.

```

llucsanchez@swarm-manager: /opt/shopmicro
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker build -t lsanchez5420/shopmicro-product-service:v2 ./product-service
[+] Building 12.5s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 150B
=> [internal] load metadata for docker.io/library/python:3.14-alpine
=> [internal] load .dockerignore
=> [internal] load dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/python:3.14-alpine@sha256:fae120f7885a06fcc9677922331391fa690d911c020abb9e8025ff3d908e510
=> => resolve docker.io/library/python:3.14-alpine@sha256:fae120f7885a06fcc9677922331391fa690d911c020abb9e8025ff3d908e510
=> [internal] load build context
=> => transferring context: 2.94kB
=> CACHED [2/4] WORKDIR /app
=> [3/4] COPY . .
=> [4/4] RUN pip install -r requirements.txt
=> => exporting layers
=> => exporting manifest sha256:b8e52bfe257d9d30481368505ae214dc355ca1a122d25d9e0adb125d215639
=> => exporting config sha256:32b151a4a5e2c09e81c8bcf2f11104be7bcf11fa9c6f75fd4d0d54a22879dc
=> => exporting attestation manifest sha256:45bb35c0b6da069fa14402a96ccaff4e08b43db99a0e0eb4877b4762296
=> => exporting manifest list sha256:b5a736abe322cfcd0093d0ce5b857887a657361344a332dc2689a2e44ef52d5
=> => naming to docker.io/lsanchez5420/shopmicro-product-service:v2
=> => unpacking to docker.io/lsanchez5420/shopmicro-product-service:v2
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker build -t lsanchez5420/shopmicro-order-service:v2 ./order-service
[+] Building 11.6s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 150B
=> [internal] load metadata for docker.io/library/python:3.14-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/python:3.14-alpine@sha256:fae120f7885a06fcc9677922331391fa690d911c020abb9e8025ff3d908e510
=> => resolve docker.io/library/python:3.14-alpine@sha256:fae120f7885a06fcc9677922331391fa690d911c020abb9e8025ff3d908e510
=> [internal] load build context
=> => transferring context: 3.62kB
=> CACHED [2/4] WORKDIR /app
=> [3/4] COPY . .
=> [4/4] RUN pip install -r requirements.txt
=> => exporting layers
=> => exporting manifest sha256:325a9a72f0307bcca02e2a2cb1ede87c34049b02794fdec1b6639165191926
=> => exporting config sha256:ebda6102e817e167e095c1f73ca8928f9907da855291668a731de39d3ad47eff
=> => exporting attestation manifest sha256:963125b3c53d48955b7cf5141c55ed2e4a700de3063c20222c9eac085481cc
=> => exporting manifest list sha256:b5a736abe322cfcd0093d0ce5b857887a657361344a332dc2689a2e44ef52d5
=> => naming to docker.io/lsanchez5420/shopmicro-order-service:v2
=> => unpacking to docker.io/lsanchez5420/shopmicro-order-service:v2
llucsanchez@swarm-manager: /opt/shopmicro$

llucsanchez@swarm-manager: /opt/shopmicro
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker build -t lsanchez5420/shopmicro-user-service:v2 ./user-service
[+] Building 10.0s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 150B
=> [internal] load metadata for docker.io/library/python:3.14-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/python:3.14-alpine@sha256:fae120f7885a06fcc9677922331391fa690d911c020abb9e8025ff3d908e510
=> => resolve docker.io/library/python:3.14-alpine@sha256:fae120f7885a06fcc9677922331391fa690d911c020abb9e8025ff3d908e510
=> [internal] load build context
=> => transferring context: 4.14kB
=> CACHED [2/4] WORKDIR /app
=> [3/4] COPY . .
=> [4/4] RUN pip install -r requirements.txt
=> => exporting layers
=> => exporting manifest sha256:800e2c7497109e4df38734308b64c031a117d5b460994c36d5c5497a4e76f9e2
=> => exporting config sha256:3eb952840d81c1fb04d12e6704bad9b509eb03c6b5e78aea61a00bd660ba5
=> => exporting attestation manifest sha256:7916a3e4d03ce1452c2206e45d0fe0b55de026bc38034adbce92c807bc927
=> => exporting manifest list sha256:b5a736abe322cfcd0093d0ce5b857887a657361344a332dc2689a2e44ef52d5
=> => naming to docker.io/lsanchez5420/shopmicro-user-service:v2
=> => unpacking to docker.io/lsanchez5420/shopmicro-user-service:v2
llucsanchez@swarm-manager: /opt/shopmicro$

```

Cop es veu a les imatges, hem posat el tag “v2” a les noves imatges, així podem controlar la versió correctament al Docker Hub, tot seguit, fem un push:

```

llucsanchez@swarm-manager: /opt/shopmicro
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker push lsanchez5420/shopmicro-product-service:v2
The push refers to repository [docker.io/lsanchez5420/shopmicro-product-service]
ff47e17be876: Pushed
de00923f05e6: Pushed
390de8e4dff6: Pushed
589002ba0eae: Layer already exists
0805a1082be0: Layer already exists
3566efde290b: Layer already exists
28000a7aef8b1: Layer already exists
7c19cf132cf5: Layer already exists
v2: digest: sha256:bea736abe322cfcd0093d0ce5b857887a657361344a332dc2689a2e44ef52d5 size: 856
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker push lsanchez5420/shopmicro-order-service:v2
The push refers to repository [docker.io/lsanchez5420/shopmicro-order-service]
0805a1082be0: Layer already exists
5dd5300d91c8: Pushed
76b812361aba: Pushed
3566efde290b: Layer already exists
28000a7aef8b1: Layer already exists
589002ba0eae: Layer already exists
7c19cf132cf5: Layer already exists
2e4b56fd5215: Pushed
v2: digest: sha256:bc8d79465d039cd26115c2f511bee581f5a9748d5a1f71415ee5fbd84049ca72 size: 856
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker push lsanchez5420/shopmicro-user-service:v2
The push refers to repository [docker.io/lsanchez5420/shopmicro-user-service]
0805a1082be0: Layer already exists
3566efde290b: Layer already exists
28000a7aef8b1: Layer already exists
7c19cf132cf5: Layer already exists
059e5b6e4630: Pushed
35ab7f5db0aa: Pushed
f44b26784bf7: Pushed
589002ba0eae: Layer already exists
v2: digest: sha256:1469dbded4e885ba30bae8d1366aa694285db919d6e21830c7bfd695932699e6 size: 856
llucsanchez@swarm-manager: /opt/shopmicro$

```

Ja ho tenim tot llest, només queda modificar l'arxiu `docker-stack.yml` amb la nova forma d'accedir a les credencials sensibles, això ho podem amb el paràmetre “secrets.”

```

1 services:
2   shop-db:
3     image: mysql:9.6
4     environment:
5       MYSQL_ROOT_PASSWORD: /run/secrets/db_root_password
6       MYSQL_DATABASE: ${DB_NAME}
7       MYSQL_USER: ${DB_USER}
8       MYSQL_PASSWORD: /run/secrets/db_password
9     secrets:
10      - db_root_password
11      - db_password
12     volumes:
13      - shop_db_data:/var/lib/mysql
14      - /opt/shopmicro/db-init/init.sql:/docker-entrypoint-initdb.d/init.sql
15     networks:
16      - backend-net
17     healthcheck:
18      test: ["CMD", "sh", "-c", "mysqladmin ping -h localhost -u root -p$(cat /run/secrets/db_root_password)"]
19      interval: 10s
20      timeout: 5s
21      retries: 5
22     deploy:
23       placement:
24         constraints: [node.role == manager]
25       restart_policy:
26         condition: on-failure

```

Com es veu a la imatge, s'utilitza la ruta “/run/secrets” per accedir a les credencials, aquesta ruta s'emmagatzema directament en memòria RAM, així si algú accedeix als discs durs del nostre sistema no podrà tindre accés les credencials.

També cal fer la declaració al final de l'arxiu:

```

secrets:
  db_root_password:
    external: true
  db_password:
    external: true

```

Ara si, ja ho tenim tot llest per tornar a desplegar la nostra solució, al esborrar tot el stack i tornar-lo a desplegar com hem fet abans.

```

llucsanchez@swarm-manager: /opt/shopmicro
llucsanchez@swarm-manager: /opt/shopmicro$ sudo docker stack rm shopmicro
(sudo) password for llucsanchez:
Removing service shopmicro_api-gateway
Removing service shopmicro_cache
Removing service shopmicro_frontend
Removing service shopmicro_message-queue
Removing service shopmicro_notification-service
Removing service shopmicro_order-service
Removing service shopmicro_product-service
Removing service shopmicro_shop-db
Removing service shopmicro_user-service
Removing network shopmicro_frontend-net
Removing network shopmicro_backend-net
llucsanchez@swarm-manager: /opt/shopmicro$ set -a; source .env; set +a
llucsanchez@swarm-manager: /opt/shopmicro$ docker stack deploy --with-registry-auth -c docker-stack.yml shopmicro
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network shopmicro_backend-net
Creating network shopmicro_frontend-net
Creating service shopmicro_shop-db
Creating service shopmicro_frontend
Creating service shopmicro_user-service
Creating service shopmicro_cache
Creating service shopmicro_api-gateway
Creating service shopmicro_product-service
Creating service shopmicro_notification-service
Creating service shopmicro_order-service
Creating service shopmicro_message-queue
llucsanchez@swarm-manager: /opt/shopmicro$

```

Si tot funciona correctament, tots els contenidors es troben en estat “Running”:

```
llucsanchez@swarm-manager:/opt/shopmicro$ docker stack ps shopmicro
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
vfvigj9yh3a8	shopmicro_api-gateway.1	lsanchez5420/shopmicro-api-gateway:v1	swarm-worker2	Running	Running about a minute ago		
qyjf86bi57yz	shopmicro_api-gateway.2	lsanchez5420/shopmicro-api-gateway:v1	swarm-worker1	Running	Running about a minute ago		
s8hczbwd6qu	shopmicro_cache.1	redis:8.6.1-alpine	swarm-manager	Running	Running about a minute ago		
ys5p6whlnp4	shopmicro_frontend.1	lsanchez5420/shopmicro-frontend:v1	swarm-worker2	Running	Running about a minute ago		
6x0kl75myxef	shopmicro_frontend.2	lsanchez5420/shopmicro-frontend:v1	swarm-worker1	Running	Running about a minute ago		
nvs5sjdcscan	shopmicro_message-queue.1	rabbitmq:4.2.5-management-alpine	swarm-manager	Running	Running about a minute ago		
6wlr704bze1	shopmicro_notification-service.1	lsanchez5420/shopmicro-notification-service:v1	swarm-worker1	Running	Running about a minute ago		
h4gsaalbc3k	shopmicro_order-service.1	lsanchez5420/shopmicro-order-service:v2	swarm-worker2	Running	Running about a minute ago		
cz24k02ndip	shopmicro_order-service.2	lsanchez5420/shopmicro-order-service:v2	swarm-worker1	Running	Running about a minute ago		
xo8ck9c4vihy	shopmicro_product-service.1	lsanchez5420/shopmicro-product-service:v2	swarm-manager	Running	Running about a minute ago		
ncbwlydxunxu	shopmicro_product-service.2	lsanchez5420/shopmicro-product-service:v2	swarm-worker2	Running	Running about a minute ago		
e9lhbq2fpt1z	shopmicro_shop-db.1	mysql:9.6	swarm-manager	Running	Running about a minute ago		
nlrns1tu7n7e	shopmicro_user-service.1	lsanchez5420/shopmicro-user-service:v2	swarm-worker2	Running	Running about a minute ago		
wglef9winda9	shopmicro_user-service.2	lsanchez5420/shopmicro-user-service:v2	swarm-worker1	Running	Running about a minute ago		

```
llucsanchez@swarm-manager:/opt/shopmicro$
```

Aïllament Xarxes overlay

En aquesta part de la Fase 3, ens hem centrat a aplicar el principi de mínim privilegi a nivell de xarxa. L'objectiu és segmentar el trànsit del clúster perquè cada servei només pugui veure i comunicar-se amb els components necessaris per al seu funcionament, reduint així la superfície d'atac en cas que un contenidor es vegi compromès.

Per complir amb els requeriments de seguretat, hem definit tres xarxes de tipus overlay diferenciades en el nostre fitxer [docker-stack.yml](#):

1. **frontend-net:** Xarxa dedicada exclusivament a la comunicació entre el client extern i el punt d'entrada de l'aplicació.
2. **api-net:** Xarxa intermèdia per a la comunicació entre el Gateway i els microserveis.
3. **backend-net:** Xarxa aïllada per a la persistència de dades, memòria cache i sistemes de missatgeria.

A continuació, detallem com s'ha implementat l'aïllament per complir els objectius fixats:

- Aïllament del Frontend

Hem configurat el servei frontend perquè estigui connectat únicament a la xarxa frontend-net. D'aquesta manera, el frontend **NOMÉS** pot parlar amb l'api-gateway (que també forma part d'aquesta xarxa). No té visibilitat directa sobre els microserveis de backend ni sobre les bases de dades.

- Comunicació dels Microserveis d'API

Els microserveis d'API (product-service, order-service i user-service) actuen com a pont entre el Gateway i les dades. Per aquest motiu, participen en dues xarxes:

api-net: Per rebre les peticions provinents de l'api-gateway.

backend-net: Per realitzar consultes a la base de dades i a la memòria cau.

Amb aquesta configuració, ens assegurem que els microserveis d'API **NOMÉS** poden parlar amb les seves BD respectives i la cache, mantenint el trànsit de dades sensible fora de les xarxes exposades a l'exterior.

- Protecció de la Capa de Dades

Els serveis crítics de persistència i infraestructura interna, com son shop-db, cache i message-queue, s'han implementat dins de la backend-net.

Com es pot comprovar en la configuració del servei, no s'ha definit cap paràmetre ports per a aquests serveis. Això garanteix que les BD no estan exposades directament a cap xarxa pública i que només són accessibles de forma interna per part dels microserveis autoritzats dins del Swarm.

```
networks:
  frontend-net:
    driver: overlay
  api-net:
    driver: overlay
  backend-net:
    driver: overlay
```

Amb aquesta estructura, hem aconseguit un entorn robust on el trànsit està totalment compartimentat, complint amb els estàndards de seguretat requerits per a un entorn de producció distribuït.

Gestió TLS Docker Swarm

Un dels punts forts de Docker Swarm és la seva gestió automatitzada de la seguretat. Per defecte, Swarm implementa **TLS** en totes les comunicacions entre els nodes del clúster. Això significa que tant les dades com el trànsit de control estan xifrats i, a més, els nodes es validen entre ells mitjançant certificats.

Quan arranquem el Swarm, el node Manager genera automàticament una **Autoritat de Certificació (CA)** interna. **Aquest procés és transparent per a l'administrador:**

- **Emissió de certificats:** Quan un nou node s'uneix al clúster, el Manager li emet un certificat signat per aquesta CA interna.
- **Comunicació xifrada:** Tota la comunicació dins del es realitza a través de canals TLS segurs.
- **Rotació:** Docker Swarm s'encarrega de la renovació automàtica d'aquests certificats

Per comprovar que el nostre node té el rol de Manager i està participant correctament en l'estructura de seguretat del Swarm, hem executat la comanda de verificació sobre el node principal:

 llucsanchez@swarm-manager: /opt/shopmicro

```
llucsanchez@swarm-manager:/opt/shopmicro$ sudo docker info | grep -A5 'Swarm'
Swarm: active
NodeID: ymf6nkcykl2a9xa2gymvzkc0x
Is Manager: true
ClusterID: zfiobiczwi79kjiwc2jwsov2
Managers: 1
Nodes: 3
llucsanchez@swarm-manager:/opt/shopmicro$
```

- **Swarm: active:** Confirma que el node forma part del mode Swarm i està operatiu.
- **Is Manager: true:** Confirma que aquest node té la capacitat de gestionar el clúster i, per tant, és el que té la clau privada de la CA interna que emet els certificats als nodes workers.

Amb aquesta configuració, garantim que qualsevol node que intenti unir-se al nostre clúster ha de presentar un token vàlid i ser validat mitjançant aquests certificats generats automàticament, protegint així tota la nostra arquitectura contra intrusions no autoritzades.

Escaneig d'imatges

Per a aquesta tasca, hem utilitzat **Docker Scout**, l'eina integrada en Docker que ens permet analitzar de forma ràpida i eficient les nostres imatges i comparar-les amb bases de dades de vulnerabilitats actualitzades.

Abans de tot, ha calgut instal·lar Docker Scout de la següent forma, com indiquen les [fonts oficials](#):

```
llucsanchez@swarm-manager: /opt/desplegament/docker-scout$ curl -fsSL https://raw.githubusercontent.com/docker/scout-cli/main/install.sh -o install-scout.sh
llucsanchez@swarm-manager: /opt/desplegament/docker-scout$ sh install-scout.sh
[info] fetching release script for tag='v1.20.4'
[info] using release tag='v1.20.4' version='1.20.4' os='linux' arch='amd64'
[info] installed /home/llucsanchez/.docker/cli-plugins/docker-scout
llucsanchez@swarm-manager: /opt/desplegament/docker-scout$
```

Tot el procés d'anàlisi d'imatges és molt senzill, amb una sola comanda hem pogut analitzar qualsevol imatge del nostre Docker Swarm, a continuació, adjuntem un vídeo demostratiu on escanegem totes les imatges que hem utilitzat per a la nostra solució:

Enllaços:

- [Youtube](#) 
- [Google Drive](#) 
- [GitHub](#)  (Documentació) **FALTA PONER**

Minutatge:

- 00:00 - 00:33 → Imatges que tenim disponibles + les que volem escanejar.
- 0:34 - 01:19 → Escaneig api-gateway
- 01:20 - 02:43 → Escaneig frontend
- 02:44 - 03:37 → Escaneig notification-service
- 03:38 - 04:22 → Escaneig order-service
- 04:23 - 05:38 → Escaneig product-service
- 05:39 - 06:18 → Escaneig user-service
- 06:19 - 07:21 → Escaneig mysql9.6
- 07:22 - 08:02 → Escaneig rabbitMQ
- 08:03 - 08:40 → Escaneig redis

Com a conclusió final, en el vídeo es pot veure que es detecten algunes vulnerabilitats importants dins de les imatges que utilitzem, una de les que més s'han repetit és la del sistema Linux "Alpine", aquest l'usem per reduir considerablement l'espai i el consum de la imatge.

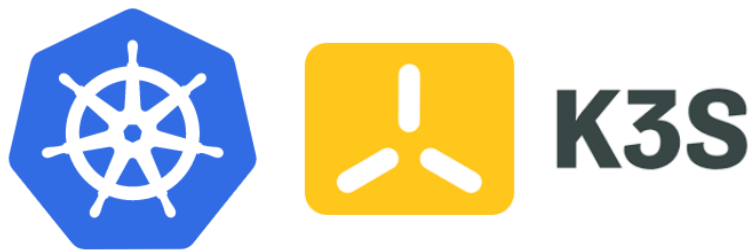
En aquest cas, no podem fer res, ja que hi ha moltíssimes imatges dins de Docker que contenen Alpine, l'única solució seria fer una imatge personalitzada d'alpine nostra + el servei que volem utilitzar o contactar amb els que han creat la imatge per notificar la vulnerabilitat.

Fase 4: Kubernetes

Introducció a Kubernetes

Després d'haver consolidat la nostra plataforma de microserveis mitjançant **Docker Compose** en un entorn de desenvolupament local i haver escalat el sistema cap a un cluster d'Alta Disponibilitat amb **Docker Swarm**, volem pujar un nivell més i migrar-ho tot a **Kubernetes**

Kubernetes ens proporciona una capacitat **d'orquestració superior**, permetent una gestió de recursos més eficient, autoreparació (self-healing) automatitzada i un ecosistema molt més extens per gestionar el desplegament de la nostra plataforma ShopMicro.



En aquesta secció **documentarem** el procés d'instal·lació del clúster des de zero, la migració de la nostra lògica de desplegament basada en fitxers docker-stack cap als objectes nadius de Kubernetes (Deployments, Services, ConfigMaps i Secrets) i la posada en marxa final del sistema, consolidant així **una infraestructura robusta, escalable** i preparada per al món real.

	Docker Swarm	Kubernetes
Escalat	Senzill, escalat ràpid amb <i>docker service scale</i>	Més complexe, en el nostre cas senzill (<i>k3s scale</i>)
Self-healing	Bàsic, reinicia contenidors amb errades, amb diagnòstic senzill.	Avançat, monitoritza estat de salut i reprograma pods amb errades.
Rolling Updates	Integrat, suficientment potent.	Integrat, amb estratègies més complexes
Gestió de Secrets	Molt fàcil i senzill d'entendre.	Molt potent, permet separar-los de la configuració i encriptar-los.
Networking	Xarxes Overlay senzilles, fàcils d'utilitzar	Més complicat, requereix Services i un Ingress.
Complexitat d'operació	Baixa, sintaxi senzilla Docker.	Alta, corba d'aprenentatge pronunciada i conceptes nous respecte a Docker

Instal·lació k3s

K3s és una distribució de Kubernetes molt coneguda per ser lleugera i certificada. L'objectiu d'aquesta és ser la meitat de Kubernetes, és a dir, només conté en el seu binari tot el necessari per executar qualsevol cosa en un clúster, el seu tamany és de menys de 100MB.

Aquest serà el que utilitzem nosaltres per l'última fase d'aquest apartat entre els 3 nodes que hem vist anteriorment (manager-worker1-worker2).

Primer de tot, com indica la [documentació oficial](#) cal introduir la següent comanda a la terminal per començar amb la instal·lació:

```
llucsanchez@swarm-manager: /opt/desplegament
llucsanchez@swarm-manager: /opt/desplegament$ curl -sfL https://get.k3s.io | sh -
[sudo] password for llucsanchez:
[INFO] Finding release for channel stable
[INFO] Using v1.34.6+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.34.6%2Bk3s1/sha256sum-amd64.txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.34.6%2Bk3s1/k3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Skipping installation of SELinux RPM
[INFO] Skipping /usr/local/bin/kubect1 symlink to k3s, already exists
[INFO] Creating /usr/local/bin/crictl symlink to k3s
[INFO] Skipping /usr/local/bin/ctr symlink to k3s, command exists in PATH at /usr/bin/ctr
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s.service
[INFO] systemd: Enabling k3s unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service -> /etc/systemd/system/k3s.service.
[INFO] systemd: Starting k3s
llucsanchez@swarm-manager: /opt/desplegament$ sudo k3s kubectl get nodes
NAME          STATUS    ROLES    AGE     VERSION
swarm-manager Ready    control-plane 3m25s   v1.34.6+k3s1
llucsanchez@swarm-manager: /opt/desplegament$
```

Aquesta executa un script de Bash que realitza tota la instal·lació automàticament.

Un cop ja el tenim dins del nostre node manager, cal obtenir el token del node amb la següent comanda (*algo similar al que vam fer amb docker-swarm*).

```
llucsanchez@swarm-manager: /opt/desplegament
llucsanchez@swarm-manager: /opt/desplegament$ sudo cat /var/lib/rancher/k3s/server/node-token
K104d123e9abe634e10c7ec2514b14da76484e8942e6d7ea5839fbc19f97674351d::server:54028b80108c0cfcb39b6bc816ee9760
llucsanchez@swarm-manager: /opt/desplegament$
```

Realitzarem el mateix dins dels nodes workers:

```
llucsanchez@swarm-worker1:~$ curl -sL https://get.k3s.io | \
  K3S_URL=https://10.0.50.10:6443 \
  K3S_TOKEN=K104d123e9abe634e10c7ec2514b14da76484e8942e6d7ea5839fbc19f97674351d::server:54028b80108c0cfc39b6bc816ee9760 \
  sh -
[INFO] Finding release for channel stable
[INFO] Using v1.34.6+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.34.6%2Bk3s1/sha256sum-amd64.txt
[INFO] Skipping binary downloaded, installed k3s matches hash
[INFO] Skipping installation of SELinux RPM
[INFO] Skipping /usr/local/bin/kubect1 symlink to k3s, already exists
[INFO] Skipping /usr/local/bin/crictl symlink to k3s, already exists
[INFO] Skipping /usr/local/bin/ctr symlink to k3s, command exists in PATH at /usr/bin/ctr
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-agent-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s-agent.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s-agent.service
[INFO] systemd: Enabling k3s-agent unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s-agent.service → /etc/systemd/system/k3s-agent.service.
[INFO] systemd: Starting k3s-agent
llucsanchez@swarm-worker1:~$
```

Quan ja ho tenim instal·lat a les 3 màquines, podem comprovar l'estat del servei utilitzant la següent comanda dins de la terminal del manager. En aquesta podem comprovar que els 3 nodes es troben operatius i preparats per funcionar:

```
llucsanchez@swarm-manager:/opt/desplegament$ sudo k3s kubectl get nodes
NAME             STATUS    ROLES    AGE   VERSION
swarm-manager    Ready    control-plane   19m   v1.34.6+k3s1
swarm-worker1    Ready    <none>         100s   v1.34.6+k3s1
swarm-worker2    Ready    <none>         11s    v1.34.6+k3s1
llucsanchez@swarm-manager:/opt/desplegament$
```

Un dels **principals problemes** que nosaltres hem experimentat durant tot aquest procés ha sigut el comportament que té k3s amb les interfícies de la màquina. Per defecte, aquest servei utilitza la primera interfície disponible que hi ha (enp0s3), com nosaltres disposem de diferents targetes de xarxa i la que ens interessa és una d'altre (enp0s8), hem d'accedir al següent fitxer i afegir els paràmetres (als 3 nodes):

```
llucsanchez@swarm-manager:~$ sudo cat /etc/systemd/system/k3s.service
[Unit]
Description=Lightweight Kubernetes
Documentation=https://k3s.io
Wants=network-online.target
After=network-online.target

[Install]
WantedBy=multi-user.target

[Service]
Type=notify
EnvironmentFile=/etc/default/%N
EnvironmentFile=/etc/sysconfig/%N
EnvironmentFile=/etc/systemd/system/k3s.service.env
KillMode=process
Delegate=yes
User=root
# Having non-zero Limit*s causes performance problems due to accounting overhead
# in the kernel. We recommend using cgroups to do container-local accounting.
LimitNOFILE=1048576
LimitNPROC=infinity
LimitCORE=infinity
TasksMax=infinity
TimeoutStartSec=0
Restart=always
RestartSec=5s
ExecStartPre=/sbin/modprobe br_netfilter
ExecStartPre=/sbin/modprobe overlay
ExecStart=/usr/local/bin/k3s \
  server \
  '--node-ip=10.0.50.10' \
  '--flannel-iface=enp0s8' \

llucsanchez@swarm-manager:~$
```

Migració d'arxius principals Docker

Amb l'anterior configuració ja ho tenim tot preparat per funcionar amb els 3 nodes alhora, ara toca migrar el nostre Docker-Compose que hem utilitzat durant la fase 1 (o *també podem migrar el Docker-Stack*), per això utilitzarem **Kompose**, una eina que te com a objectiu convertir els orquestradors de Docker a format Kubernetes.

Segons la [web oficial](#), per realitzar la instal·lació d'aquesta eina només cal introduir una comanda per la terminal, igual que hem fet amb k3s:

```
llucsanchez@swarm-manager:~/Descargas$ curl -L https://github.com/kubernetes/kompose/releases/download/v1.38.0/kompose-linux-amd64 -o kompose
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0:00:00 0:00:00 0:00:00 0
25 24.2M 25 6446k 0 0 1924k 0 0:00:12 0:00:03 0:00:09 2349k
```

Un cop tenim el binari que hem descarregat, l'hem de moure dins de la carpeta de binaris de Linux, així el podem executar directament des de la terminal:

```
llucsanchez@swarm-manager: ~/Descargas

llucsanchez@swarm-manager:~/Descargas$ sudo chmod +x kompose
llucsanchez@swarm-manager:~/Descargas$ sudo mv ./kompose /usr/local/bin/kompose
llucsanchez@swarm-manager:~/Descargas$

llucsanchez@swarm-manager: ~/Descargas

llucsanchez@swarm-manager:~/Descargas$ kompose
Kompose is a tool to help users who are familiar with docker-compose move to Kubernetes.

Usage:
  kompose [command]

Examples:
  kompose --file compose.yaml convert
  kompose -f first.yaml -f second.yaml convert
  kompose --provider openshift --file compose.yaml convert
  kompose completion bash

Available Commands:
  completion  Output shell completion code
  convert      Convert a Compose file
  help        Help about any command
  version     Print the version of Kompose

Flags:
  --error-on-warning  Treat any warning as an error
  -f, --file strings  Specify an alternative compose file
  -h, --help          help for kompose
  --provider string   Specify a provider. Kubernetes or OpenShift. (default "kubernetes")
  --suppress-warnings Suppress all warnings
  -v, --verbose       verbose output

Use "kompose [command] --help" for more information about a command.
llucsanchez@swarm-manager:~/Descargas$
```

Amb la comanda **kompose convert**, podem passar-li el fitxer docker-compose i ens realitza tota la conversió:

```
-rw-r--r-- 1 llucsanchez llucsanchez 2981 Apr 21 16:31 docker-compose_v2.yml
llucsanchez@swarm-manager:/opt/desplegament_v2$ kompose convert -f docker-compose_v2.yml --namespace shopmicro3
WARN The "DB_HOST" variable is not set. Defaulting to a blank string.
```

Quan executem la comanda, ens indica els **possibles errors que ha trobat al fer la conversió**, en el nostre cas els més crítics son les variables d'entorn, els fitxers service i els configmaps.

```
llucsanchez@swarm-manager:/opt/desplegament_v2$ ls -l
total 4
-rw-r--r-- 1 llucsanchez llucsanchez 2901 Apr 21 16:31 docker-compose_v2.yml
llucsanchez@swarm-manager:/opt/desplegament_v2$ kompose convert -f docker-compose_v2.yml --namespace shopmicro3
WARN The "DB_HOST" variable is not set. Defaulting to a blank string.
WARN The "DB_USER" variable is not set. Defaulting to a blank string.
WARN The "DB_PASSWORD" variable is not set. Defaulting to a blank string.
WARN The "DB_NAME" variable is not set. Defaulting to a blank string.
WARN The "REDIS_HOST" variable is not set. Defaulting to a blank string.
WARN The "RMQ_HOST" variable is not set. Defaulting to a blank string.
WARN The "DB_HOST" variable is not set. Defaulting to a blank string.
WARN The "DB_USER" variable is not set. Defaulting to a blank string.
WARN The "DB_PASSWORD" variable is not set. Defaulting to a blank string.
WARN The "DB_NAME" variable is not set. Defaulting to a blank string.
WARN The "REDIS_HOST" variable is not set. Defaulting to a blank string.
WARN The "DB_ROOT_PASSWORD" variable is not set. Defaulting to a blank string.
WARN The "DB_USER" variable is not set. Defaulting to a blank string.
WARN The "DB_PASSWORD" variable is not set. Defaulting to a blank string.
WARN The "DB_NAME" variable is not set. Defaulting to a blank string.
WARN The "REDIS_HOST" variable is not set. Defaulting to a blank string.
WARN The "RMQ_HOST" variable is not set. Defaulting to a blank string.
WARN /opt/desplegament_v2/docker-compose_v2.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
WARN Restart policy 'unless-stopped' in service user-service is not supported, convert it to 'always'
WARN Restart policy 'unless-stopped' in service shop-db is not supported, convert it to 'always'
WARN Restart policy 'unless-stopped' in service notification-service is not supported, convert it to 'always'
WARN Restart policy 'unless-stopped' in service product-service is not supported, convert it to 'always'
WARN Restart policy 'unless-stopped' in service frontend is not supported, convert it to 'always'
WARN Restart policy 'unless-stopped' in service api-gateway is not supported, convert it to 'always'
WARN Restart policy 'unless-stopped' in service message-queue is not supported, convert it to 'always'
WARN Restart policy 'unless-stopped' in service cache is not supported, convert it to 'always'
WARN Restart policy 'unless-stopped' in service order-service is not supported, convert it to 'always'
WARN File don't exist or failed to check if the directory is empty: stat /var/log/nginx: no such file or directory
WARN Service "frontend" won't be created because 'ports' is not specified
WARN File don't exist or failed to check if the directory is empty: stat /var/lib/rabbitmq: no such file or directory
WARN Service "notification-service" won't be created because 'ports' is not specified
WARN Service "order-service" won't be created because 'ports' is not specified
WARN Service "product-service" won't be created because 'ports' is not specified
WARN Service "shop-db" won't be created because 'ports' is not specified
WARN File don't exist or failed to check if the directory is empty: stat /var/lib/mysql: no such file or directory
WARN File don't exist or failed to check if the directory is empty: stat /opt/desplegament_v2/db-init/init.sql: no such file or directory
WARN Volume mount on the host "/opt/desplegament_v2/db-init/init.sql" isn't supported - ignoring path on the host
WARN Service "user-service" won't be created because 'ports' is not specified
INFO Kubernetes file "api-gateway-service.yaml" created
INFO Kubernetes file "message-queue-service.yaml" created
INFO Kubernetes file "shopmicro3-namespace.yaml" created
```

Variables d'entorn → Les migrarem amb el sistema "Secrets" que té Kubernetes.

Fitxers Service → Els crearem manualment, en aquests indicarem el port i protocol que utilitza cada servei.

Configmaps → Fitxers de configuració que hem perdut o volem personalitzar (com el script d'inici de la BBDD)

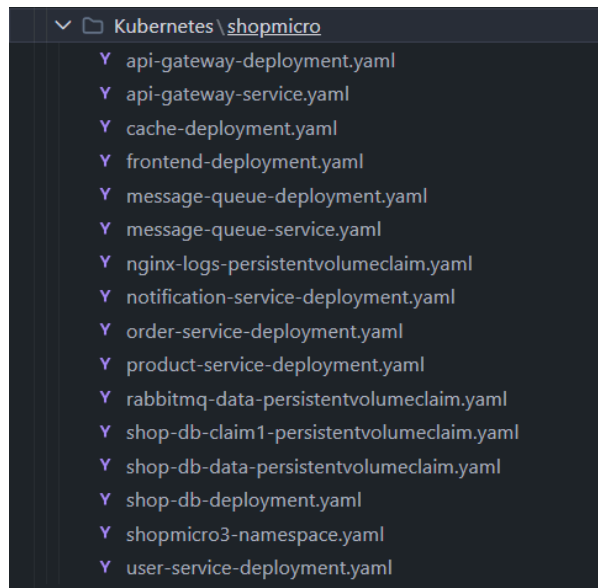
Al final del tot ens apareixen tots els arxius que ha creat Kompose:

```
INFO Kubernetes file "api-gateway-service.yaml" created
INFO Kubernetes file "message-queue-service.yaml" created
INFO Kubernetes file "shopmicro3-namespace.yaml" created
INFO Kubernetes file "api-gateway-deployment.yaml" created
INFO Kubernetes file "nginx-logs-persistentvolumeclaim.yaml" created
INFO Kubernetes file "cache-deployment.yaml" created
INFO Kubernetes file "frontend-deployment.yaml" created
INFO Kubernetes file "message-queue-deployment.yaml" created
INFO Kubernetes file "rabbitmq-data-persistentvolumeclaim.yaml" created
INFO Kubernetes file "notification-service-deployment.yaml" created
INFO Kubernetes file "order-service-deployment.yaml" created
INFO Kubernetes file "product-service-deployment.yaml" created
INFO Kubernetes file "shop-db-deployment.yaml" created
INFO Kubernetes file "shop-db-data-persistentvolumeclaim.yaml" created
INFO Kubernetes file "shop-db-claim1-persistentvolumeclaim.yaml" created
INFO Kubernetes file "user-service-deployment.yaml" created
llucsanchez@swarm-manager:/opt/desplegament_v2$
```

En la imatge es veu que s'han creat els arxius deployment i persistentvolumeclaim (volums).

Configuració final Kubernetes

Ara, ja tenim la primera part de la configuració feta, disposem d'una bona base per poder continuar amb el nostre desplegament a Kubernetes.

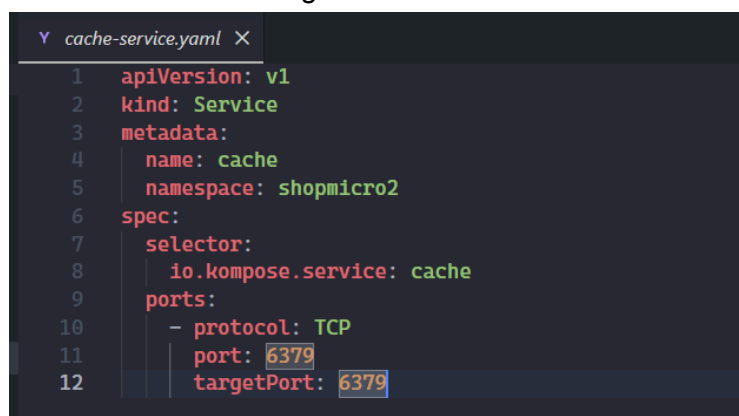


- Arxius Service

Com hem comentat anteriorment, cal definir els arxius **service** per indicar-li a Kubernetes quins ports utilitza cada microservei i així permetre que els serveis es puguin comunicar entre ells.

Els arxius service els configurarem de la mateixa manera que al Docker-Swarm, indicarem el port i protocol que utilitza cada microservei.

Aquests arxius son molt semblants al següent:



Cal realitzar un service per cada microservei que necessiti un port per comunicar-se.

Tots aquests arxius es poden consultar al nostre [GitHub](#).

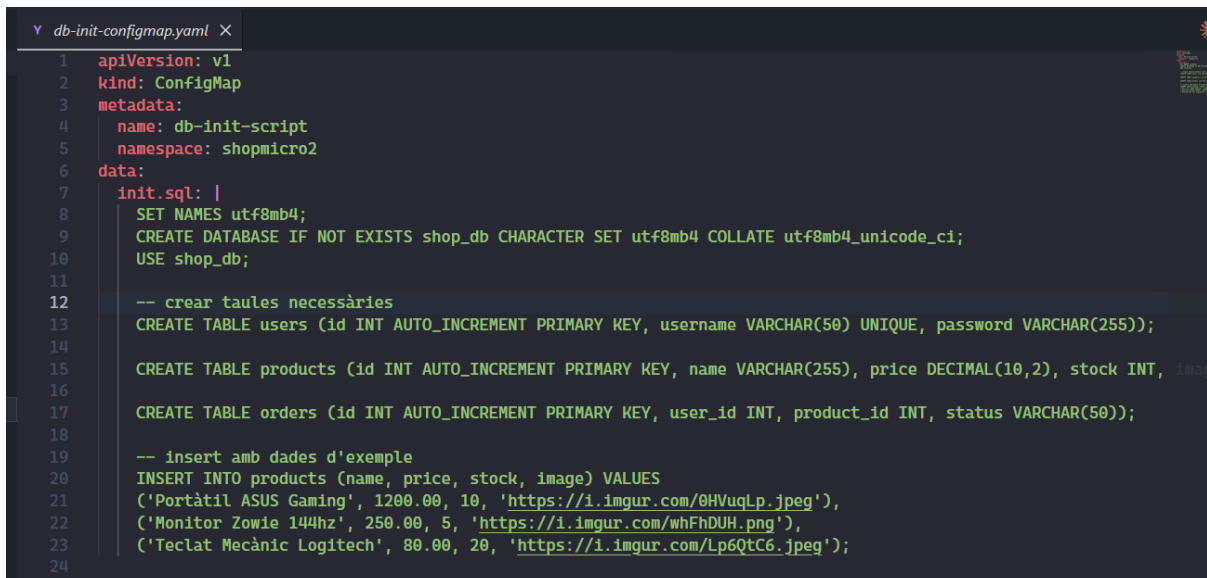
- Arxius configmap

Per mantenir alguns desplegaments que depenen d'un servei, cal crear un tipus d'arxiu anomenat "configmap", en aquest s'introdueix una configuració sencera per un microservei, per exemple, per el servei de nginx podem definir un configmap on hi ha tot l'arxiu nginx.conf. Nosaltres l'hem utilitzat per no tindre que modificar la imatge de docker que hem fet anteriorment en altres fases.

En el nostre desplegament utilitzarem configmaps per els següents objectius:

- Modificar la configuració de Nginx sense tocar la imatge de Docker (nginx.conf)
- Afegir l'arxiu inicial de creació de BBDD (init.sql)
- Passar variables d'entorn dins del nostre desplegament (.env)

Exemple configmap BBDD:



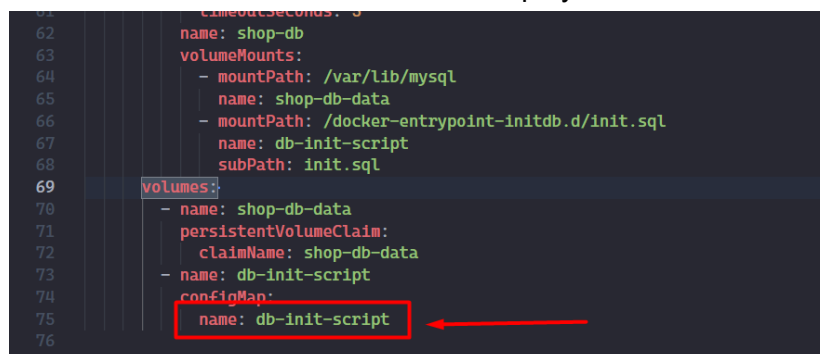
```

1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: db-init-script
5    namespace: shopmicro2
6  data:
7    init.sql: |
8      SET NAMES utf8mb4;
9      CREATE DATABASE IF NOT EXISTS shop_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
10     USE shop_db;
11
12     -- crear taules necessàries
13     CREATE TABLE users (id INT AUTO_INCREMENT PRIMARY KEY, username VARCHAR(50) UNIQUE, password VARCHAR(255));
14
15     CREATE TABLE products (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(255), price DECIMAL(10,2), stock INT, image VARCHAR(255));
16
17     CREATE TABLE orders (id INT AUTO_INCREMENT PRIMARY KEY, user_id INT, product_id INT, status VARCHAR(50));
18
19     -- insert amb dades d'exemple
20     INSERT INTO products (name, price, stock, image) VALUES
21     ('Portàtil ASUS Gaming', 1200.00, 10, 'https://i.imgur.com/0HVuqLp.jpeg'),
22     ('Monitor Zowie 144hz', 250.00, 5, 'https://i.imgur.com/whFhDUH.png'),
23     ('Teclat Mecànic Logitech', 80.00, 20, 'https://i.imgur.com/Lp6QtC6.jpeg');
24

```

Com es veu a la imatge, hi ha un arxiu amb format .sql, aquest és el que hem utilitzat en anteriors fases per generar la BBDD.

Aquests arxius cal definir-los correctament dins del deployment de cada microservei:



```

62     name: shop-db
63     volumeMounts:
64       - mountPath: /var/lib/mysql
65         name: shop-db-data
66       - mountPath: /docker-entrypoint-initdb.d/init.sql
67         name: db-init-script
68         subPath: init.sql
69     volumes:
70       - name: shop-db-data
71         persistentVolumeClaim:
72           claimName: shop-db-data
73       - name: db-init-script
74         configMap:
75           name: db-init-script
76

```

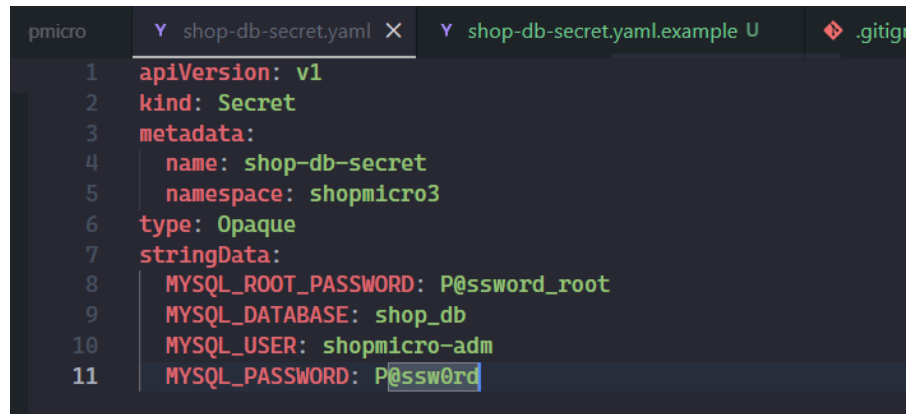
Els altres arxius es poden consultar al nostre [GitHub](#).

- Kubernetes Secrets

Aquesta tècnica serveix per poder emmagatzemar contrasenyes de forma encriptada dins de Kubernetes, amb Docker funciona exactament igual, en el nostre cas hem trobat 2 formes per implementar-ho.

1. Crear un arxiu específic per secrets

Per poder gestionar les credencials de la BBDD i no pujar-les a GitHub, hem creat l'arxiu **shop-db-secret.yaml**, on es troben les contrasenyes d'accés per a que els altres microserveis puguin obtenir les credencials sense problema.



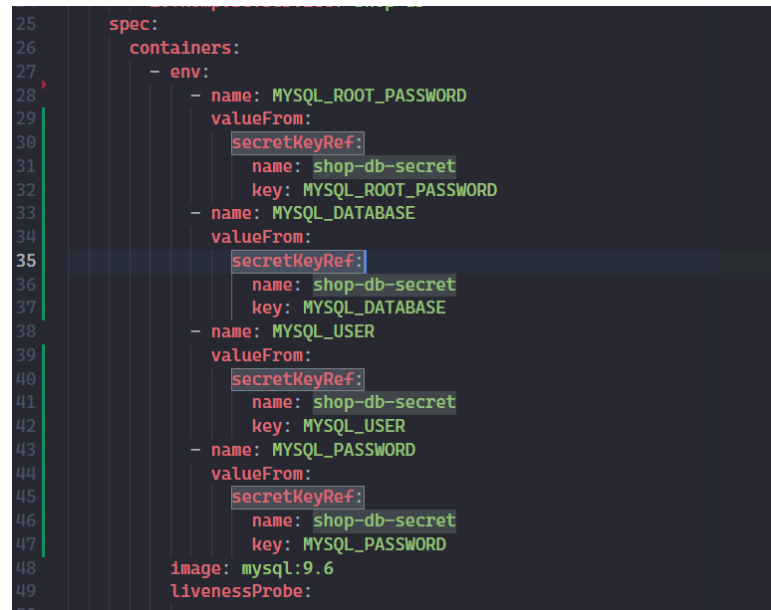
```

1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: shop-db-secret
5    namespace: shopmicro3
6  type: Opaque
7  stringData:
8    MYSQL_ROOT_PASSWORD: P@ssword_root
9    MYSQL_DATABASE: shop_db
10   MYSQL_USER: shopmicro-adm
11   MYSQL_PASSWORD: P@ssw0rd

```

Per el nostre entorn de proves, aquesta solució funciona perfectament, en un entorn real empresarial utilitzaríem un gestor de secrets extern, com AWS o Vault, així es mantenen les credencials externament.

Igual que abans, cap adaptar els arxius deployment per aquesta nova configuració:



```

25  spec:
26    containers:
27      - env:
28        - name: MYSQL_ROOT_PASSWORD
29          valueFrom:
30            secretKeyRef:
31              name: shop-db-secret
32              key: MYSQL_ROOT_PASSWORD
33        - name: MYSQL_DATABASE
34          valueFrom:
35            secretKeyRef:
36              name: shop-db-secret
37              key: MYSQL_DATABASE
38        - name: MYSQL_USER
39          valueFrom:
40            secretKeyRef:
41              name: shop-db-secret
42              key: MYSQL_USER
43        - name: MYSQL_PASSWORD
44          valueFrom:
45            secretKeyRef:
46              name: shop-db-secret
47              key: MYSQL_PASSWORD
48      image: mysql:9.6
49      livenessProbe:
50

```

2. Utilitzar una comanda

Per crear secrets dins de Kubernetes, també es pot utilitzar la següent comanda:

```
sudo k3s kubectl create secret generic shop-db-secret \
  --from-literal=MYSQL_ROOT_PASSWORD=P@ssw0rd_root \
  --from-literal=MYSQL_DATABASE=shop-db \
  --from-literal=MYSQL_USER=shopmicro-adm \
  --from-literal=MYSQL_PASSWORD=P@ssw0rd \
  -n shopmicro3
```

- Volums de Kubernetes

A diferència de Docker Swarm, on definíem el **driver NFS** directament en la configuració del servei, en aquesta fase de Kubernetes hem implementat **PersistentVolumeClaims (PVC)**.

Aquesta decisió s'ha pres per seguir les bones pràctiques de Kubernetes, on es desacobla la definició de l'emmagatzematge de la definició de l'aplicació. Això ens permet gestionar el cicle de vida de les dades (persistència, polítiques de recuperació i quotes) de manera independent al cicle de vida del Pod, garantint una major resiliència i facilitant la portabilitat de la nostra plataforma cap a entorns Cloud.

Problemes

Ens hem trobat molts de problemes durant la nostra migració a Kubernetes, els que més ens ha costat en donar-nos compte són:

- Canvi DNS a Nginx

Hem modificat els DNS que utilitzàvem a Nginx (proxy-invers) de Docker a noms de domini que Kubernetes pot resoldre amb el seu DNS Intern.

```
# Variables per resoldre dinàmicament (en temps d'execució)
set $frontend "frontend.shopmicro2.svc.cluster.local:80";
set $product "product-service.shopmicro2.svc.cluster.local:5001";
set $order "order-service.shopmicro2.svc.cluster.local:5002";
set $user "user-service.shopmicro2.svc.cluster.local:5003";

# Enrutament de peticions a cada servei

# Ruta arrel - Serveix el frontend estàtic
location / { proxy_pass http://$frontend; }

# Ruta API de productes - Permet tant /api/products com /api/products/123
location ~ ^/api/products(/.*)?$ { proxy_pass http://$product/products$1; }

# Ruta API de comandes - Permet tant /api/orders com /api/orders/123
location ~ ^/api/orders(/.*)?$ { proxy_pass http://$order/orders$1; }

# Ruta API d'usuaris - Permet tant /api/users com /api/users/login
location ~ ^/api/users(/.*)?$ { proxy_pass http://$user/users$1; }
```

Gràcies al configmap ho hem pogut fer sense tocar la imatge de Docker.

- Canvi Codi PHP al Frontend

Igual que hem fet amb Nginx, hem modificat els noms DNS perquè Kubernetes els pugui resoldre:

```
1 <?php
2 session_start();
3 if(isset($_SESSION['user_id'])) { header('Location: index.php'); exit; }
4 $error = '';
5 if($_SERVER['REQUEST_METHOD'] == 'POST') {
6     $data = json_encode(['username' => $_POST['username'], 'password' => $_POST['password']]);
7     $options = [
8         'http' => ['header' => "Content-type: application/json\r\n", 'method' => 'POST', 'content' => $data],
9         'ssl' => ['verify_peer' => false, 'verify_peer_name' => false]
10    ];
11    $context = stream_context_create($options);
12    $result = file_get_contents('https://api-gateway.shopmicro2.svc.cluster.local/api/users/login', false, $context);
13    if($result) {
14        $res = json_decode($result, true);
15        if($res['status'] == 'OK') {
16            $_SESSION['user_id'] = $res['user']['id'];
17            $_SESSION['username'] = $res['user']['username'];
18            header('Location: index.php'); exit;
19        } else { $error = $res['message']; }
20    } else { $error = "API Error: " . $res['message']; }
21 }
22
23 <!DOCTYPE html>
```

He modificat la imatge de Docker amb un nou tag "shopmicro-frontend:kubernetes"

- Utilització de Port Forwarding per accedir

L'accés a la web des de Docker és molt menys complexe que amb Kubernetes. Hem tingut que utilitzar Port-Forwarding per poder accedir a la web des d'un ordinador extern.

Aquesta tècnica crea un túnel per on viatgen les dades directament cap als pods, això només ho utilitzem en entorn de proves. En un entorn real s'utilitza un Ingress o un LoadBalancer (Cloud) per gestionar-ho correctament.

```

llucsanchez@swarm-manager: ~
Last login: Mon May  4 15:09:06 2026 from 192.168.10.1
llucsanchez@swarm-manager:~$ sudo k3s kubectl port-forward -n shopmicro2 svc/api
-gateway 8080:80 8443:443 --address 0.0.0.0
Forwarding from 0.0.0.0:8080 -> 80
Forwarding from 0.0.0.0:8443 -> 443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8443
Handling connection for 8080

```

Actualització: Hem aconseguit resoldre aquest problema utilitzant NodePorts, s'encarreguen de "mapejar" els ports que tenim oberts internament a uns d'altres externs per a que puguem accedir nosaltres, en el nostre cas, hem redirigit els ports 80 i 443 cap a 30080 i 30443:

```

api-gateway-deployment.yaml  frontend-deployment.yaml  frontend-service.yaml  api-gateway-service.yaml M X  ap
1  apiVersion: v1
2  kind: Service
3  metadata:
4    annotations:
5      kompose.cmd: kompose convert -f docker-compose_v2.yml --namespace shopmicro3
6      kompose.version: 1.38.0 (a8f5d1cbd)
7    labels:
8      io.kompose.service: api-gateway
9    name: api-gateway
10   namespace: shopmicro3
11  spec:
12   type: NodePort
13   selector:
14     io.kompose.service: api-gateway
15   ports:
16     - name: "80"
17       port: 80
18       targetPort: 80
19       nodePort: 30080
20     - name: "443"
21       port: 443
22       targetPort: 443
23       nodePort: 30443
24

```

Ara, podem accedir a la web: <https://shopmicro.local:30443>

- Pre-Carregament d'imatges a Kubernetes

Ens hem trobat que en molts casos no podem utilitzar la imatge de Docker directament des de DockerHub, l'hem d'instal·lar manualment a Kubernetes.

Això ha sigut causat per una saturació de peticions de la nostra IP Pública de l'institut que no ens permetia fer un pull d'aquestes imatges alhora d'arrencar els serveis.

Per poder precarregar una imatge a Kubernetes (s'ha de fer als 3 nodes):

```
# Pujar Imatge a DockerHub (per tindre Backup)
sudo docker build -t lsanchez5420/shopmicro-frontend:kubernetes .
sudo docker push lsanchez5420/shopmicro-frontend:kubernetes # Només node manager

# Carregar la imatge a Kubernetes
sudo docker save lsanchez5420/shopmicro-frontend:kubernetes > /tmp/shopmicro-frontend.tar
sudo k3s ctr images import /tmp/shopmicro-frontend.tar
sudo k3s kubectl rollout restart deployment/frontend -n shopmicro2

# Reniciar.....
```

Desplegament Final / Video Demostratiu

Un cop ja ho tenim tot preparat i correctament configurat, podem començar amb el desplegament als 3 nodes de la nostra web.

Cal deixar tots els arxius que hem anat creant durant aquesta fase dins d'una carpeta en la que tinguem permisos de lectura i escriptura.

```
llucsanchez@swarm-manager: /opt
llucsanchez@swarm-manager:/opt$ tree desplegament_v2/
desplegament_v2/
├── api-gateway-configmap.yaml
├── api-gateway-deployment.yaml
├── api-gateway-service.yaml
├── cache-deployment.yaml
├── cache-service.yaml
├── db-init-configmap.yaml
├── frontend
│   ├── Dockerfile
│   ├── index.html
│   ├── index.php
│   ├── login.php
│   ├── logout.php
│   ├── register.php
│   └── styles
│       ├── index.css
│       ├── login.css
│       ├── register.css
│       ├── success.css
│       └── success.php
├── frontend-deployment.yaml
├── frontend-service.yaml
├── message-queue-deployment.yaml
├── message-queue-service.yaml
├── nginx-logs-persistentvolumeclaim.yaml
├── notification-service-deployment.yaml
├── notification-service-service.yaml
├── order-service-deployment.yaml
├── order-service-service.yaml
├── product-service-deployment.yaml
├── product-service-service.yaml
├── rabbitmq-data-persistentvolumeclaim.yaml
├── service-configmap.yaml
├── shop-db-claim1-persistentvolumeclaim.yaml
├── shop-db-data-persistentvolumeclaim.yaml
├── shop-db-deployment.yaml
├── shop-db-secret.yaml
├── shop-db-secret.yaml.example
├── shop-db-service.yaml
├── shopmicro3-namespace.yaml
├── user-service-deployment.yaml
└── user-service-service.yaml

3 directories, 39 files
llucsanchez@swarm-manager:/opt$
```

Tots aquests arxius es poden obtenir directament del nostre [GitHub](#).

A continuació, mostrem un video amb tota la demostració del desplegament des de 0, demostrem els següents apartats:

- Posada en marxa desplegament Kubernetes des de 0 (GitHub)
- Accés a la web i comprovar els fluxes.
- Veure logs dels serveis (pods)
- Parada d'un Pod en calent.
- Actualització d'imatge Docker en calent.
- Augment de rèpliques a 4

Enllaços:

- [Youtube](#) 
- [Google Drive](#) 
- [GitHub](#)  (Documentació) **FALTA PONER**

Minutatge:

- 00:00 - 00:14 → GitHub del projecte + estat actual màquines a VirtualBox.
- 00:15 - 00:43 → Clonació GitHub del projecte al Manager.
- 00:44 - 01:01 → Moure arxiu secrets del nostre desplegament de proves. (no penjat a Github)
- 01:02 - 01:20 → Veure nodes actius a Kubernetes (Manager-Worker1-Worker2).
- 01:21 - 01:36 → Moure arxiu secrets al directori pertinent de Kubernetes.
- 01:37 - 01:44 → Creació namespace "shopmicro3".
- 01:45 - 02:00 → Iniciar desplegament / crear pods amb comanda "apply".
- 02:01 - 02:35 → Comprovar que els pods es troben actius.
- 02:36 - 03:35 → Accés a la web + comprovació fluxes. (fer comandes, accés usuari)
- 03:36 - 04:20 → Comprovar logs de RabbitMQ.
- 04:21 - 05:10 → Comprovar serveis actius al desplegament.
- 05:11 - 06:15 → Comanda "describe" per veure logs més detallats del pod.
- 06:16 - 06:31 → Demostració NIC del Manager.
- 06:32 - 07:58 → Canvi d'imatge en calent al product-service. (pujada de versió v2 a v3)
- 07:59 - 08:37 → Accés a la web de nou.
- 08:38 - 09:10 → Comanda "describe" per comprovar que s'ha pujat la imatge
- 09:11 - 10:06 → Veure el deployment.yaml per comprovar la imatge original.
- 10:07 - 11:39 → Esborrar el pod i comprovar que arranca de nou automàticament.
- 11:40 - 12:29 → Augment a 4 rèpliques en un pod.